

Sweet Bites

Loja de Cupcakes Gourmet
Documentação do projeto

Projeto Integrador Transdisciplinar em Engenharia de Software II

Bruno Villanova da Silva

Repositório: github.com/Brunoredbov/sweet-bites

Sumário

1. Parte 1: Documentação revisada (Situação-Problema 1)
    1. Situação-Problema 1: Documentação revisada
    2. 1. Visão geral do projeto
    3. 2. Requisitos: histórias de usuário (revisadas)
    4. 3. Product Backlog e Sprints
    5. 4. Casos de uso (revisados)
    6. 5. Diagrama de classes (revisado)
    7. 6. Diagramas comportamentais (revisados)
    8. 7. Projeto físico do banco de dados
    9. 8. Arquitetura da solução
    10. 9. Protótipos e telas implementadas
    11. 10. Registro de revisões (melhoria contínua)
2. Parte 2: Codificação e manual de uso (Situação-Problema 2)
    1. Sweet Bites: código da aplicação (Situação-Problema 2)
3. Parte 3: Testes, validação e qualidade (Situação-Problema 3)
    1. Situação-Problema 3: Testes, validação e qualidade
    2. 1. Plano de testes
    3. 2. Testes automatizados (verificação)
    4. 3. Roteiro para os testadores (validação com usuários)
    5. 4. Formulário de feedback dos testadores
    6. 5. Resultados dos testes de aceitação
    7. 6. Laudo de qualidade

Parte 1: Documentação revisada (Situação-Problema 1)

Situação-Problema 1: Documentação revisada

Revisão da documentação do **PIT I** (aplicativo Sweet Bites), com as atualizações e melhorias feitas no PIT II. Os diagramas usam Mermaid e aparecem desenhados direto no GitHub.

#	Documento	Conteúdo
1	Visão geral	Problema, objetivo, atores, escopo e decisão de plataforma
2	Requisitos	21 histórias de usuário revisadas, com CA, RN, RNF e status
3	Backlog e sprints	Backlog priorizado, pontos, andamento e definição de pronto
4	Casos de uso	Diagrama geral e casos expandidos UC-08, UC-01 e UC-16
5	Diagrama de classes	Modelo de domínio corrigido + tabela de correções
6	Sequência e estados	Realizar pedido, ciclo de vida do pedido e recuperar senha
7	Banco de dados	DER, dicionário de dados, restrições e índices (PostgreSQL)
8	Arquitetura	MVC, tecnologias, implantação, segurança e integrações
9	Protótipos e telas	Mapa navegacional e telas reais do sistema
10	Registro de revisões	O que mudou desde o PIT I e por quê

Os artefatos originais do PIT I estão em `imagens/pit1-original/`.

1. Visão geral do projeto

Problema

A **Sweet Bites** é uma loja de cupcakes gourmet que vendia apenas no balcão e por mensagens. Os pedidos ficavam espalhados em conversas, sem controle de estoque. Também não havia como o cliente ver o cardápio atualizado ou acompanhar a entrega (briefing do PIT I).

Objetivo

Oferecer uma **loja virtual mobile-first** em que o cliente:

- consulta o cardápio com fotos e preços;
- monta o carrinho e faz o pedido com entrega;
- paga com cartão, PIX ou dinheiro;
- acompanha o status do pedido em tempo real.

O administrador mantém o cardápio, o estoque e os pedidos num painel próprio.

Partes interessadas (stakeholders)

Ator	Papel no sistema
Cliente	Navega pela vitrine, compra e acompanha pedidos
Administrador	Gerencia cupcakes, categorias, estoque e pedidos
Sistema de Pagamento (externo)	Processa cartão e PIX. **No PIT II é simulado** (ver Arquitetura)
Serviço de CEP (externo)	ViaCEP: valida o endereço de entrega

Escopo

Entregue no PIT II (Sprints 1 a 3)	Fica para a evolução (Sprint 4)
Cadastro, login, logout, recuperação e troca de senha	US-11 Avaliar pedido
Vitrine, busca, detalhes, carrinho	US-12 Histórico com "Repetir pedido"
Pedido com frete por CEP, pagamento, acompanhamento	US-13 Favoritos
Admin: produtos, estoque, pedidos, categorias	US-17 Relatórios de vendas em PDF

> A tela "Meus pedidos" já lista os pedidos do cliente com status e valor, mas sem a opção "Repetir pedido". Essa opção completa a US-12 na Sprint 4.

Decisão de plataforma: de "app nativo" para "web app mobile-first"

O PIT I descrevia um **aplicativo mobile**. No PIT II a solução virou uma **aplicação web responsiva com layout de celular**. Os motivos foram:

1. **Exigências da disciplina:** o material pede front-end em HTML, CSS e JavaScript, back-end em Java/PHP/C#/Python no padrão MVC e um **link da solução em funcionamento** para os colegas testarem.
2. **Uso imediato pelos testadores:** basta um link no navegador do celular, sem instalar APK nem passar por loja de aplicativos.
3. **Fidelidade ao design:** os mockups do PIT I (rosa e branco, navegação inferior, cards) foram mantidos, então no celular a experiência é a de um app.

2. Requisitos: histórias de usuário (revisadas)

As histórias seguem o formato *"Como [papel], eu quero [ação], para que [benefício]"*, com critérios de aceitação (CA), regras de negócio (RN) e requisitos não funcionais (RNF).

Legenda de status:

- ✅ implementada;
- 🟡 implementada com ajuste (explicado em **Revisão PIT II**);
- ⏳ planejada para a Sprint 4.

O que mudou em relação ao PIT I: regras ambíguas foram esclarecidas, pontos que dependiam de serviços pagos foram ajustados (pagamento, e-mail, push) e entraram duas histórias novas (US-19 e US-20), identificadas durante o desenvolvimento.

Autenticação

US-01: Cadastrar conta ✅ · Alta · 3 pts

Ação: Como um cliente não cadastrado, eu quero criar uma conta com nome, e-mail, senha e telefone, para que eu possa acessar o aplicativo e realizar pedidos.

- CA#1: O sistema valida o e-mail como único (sem diferenciar maiúsculas/minúsculas).
- CA#2: A senha deve ter no mínimo 8 caracteres, com letras e números.
- CA#3: Um e-mail de boas-vindas é enviado após o cadastro.
- CA#4 *(novo)*: O telefone precisa ter DDD, ex.: (11) 91234-5678.
- RN#1: E-mail único no sistema. RN#2: Senha mínima de 8 caracteres.
- RNF#1: Cadastro processado em menos de 3 segundos.

Revisão PIT II: o e-mail é **simulado** (registrado no log). As mensagens de erro aparecem embaixo de cada campo, uma por campo.

US-02: Fazer login ✅ · Alta · 2 pts

Ação: Como um cliente cadastrado, eu quero fazer login com meu e-mail e senha, para que eu possa acessar minha conta e meus pedidos.

- CA#1: Autentica em até 3 segundos. CA#2: Opção "Esqueci minha senha" disponível.
- RN#1: Bloquear a conta após 5 tentativas erradas.
- RN#2 *(novo)*: A conta bloqueada é desbloqueada ao redefinir a senha (US-18).
- RNF#1: Tempo de resposta máximo de 2 segundos.

Revisão PIT II: o administrador usa a mesma tela de login e é levado direto ao painel. Quem já está logado e abre a tela de login é levado à vitrine. Com a sessão expirada, o sistema volta ao login com o aviso "Sua sessão foi encerrada" (BUG-06).

US-21: Alterar senha (logado) ✅ · Média · 2 pts · *nova no PIT II*

Ação: Como um cliente logado, eu quero trocar minha senha informando a senha atual, para não precisar do fluxo de "Esqueci minha senha".

- RN#1: A senha atual precisa estar correta. RN#2: A nova senha segue a regra de 8 caracteres ou mais, com letras e números, e deve ser diferente da atual.

Origem: revisão de qualidade pós-aceitação (BUG-08 do laudo).

US-03: Fazer logout ✅ · Alta · 1 pt

Ação: Como um cliente autenticado, eu quero fazer logout, para que minha sessão seja encerrada com segurança.

- CA#1: Após o logout, o usuário volta à tela de login.
- RN#1: A sessão é invalidada no servidor.

US-18: Recuperar senha 🟡 · Alta · 3 pts

Ação: Como um cliente que esqueceu a senha, eu quero solicitar a recuperação por e-mail, para que eu recupere o acesso à minha conta.

- CA#1: E-mail com link de redefinição. CA#2: O link expira em 1 hora e só pode ser usado uma vez.
- RN#1: Após a redefinição, todas as sessões ativas do usuário são encerradas.
- RN#2 *(novo)*: A resposta é a mesma exista ou não o e-mail (não revela quem é cliente).
- RNF#1: E-mail enviado em menos de 1 minuto.

Revisão PIT II: o e-mail é simulado. No **modo demonstração**, o link aparece na própria tela para os testadores conseguirem concluir o fluxo.

Vitrine

US-04: Navegar pela vitrine ✅ · Alta · 5 pts

Ação: Como um cliente autenticado, eu quero visualizar o cardápio com fotos, nomes, preços e categorias, para que eu escolha os produtos.

- CA#1: Cupcakes em grade (2 colunas no celular) com foto, nome e preço. CA#2: Filtro por categoria.
- RN#1: Cupcakes sem estoque aparecem como "Indisponível", com o botão desabilitado.
- RNF#1: Imagens carregam em menos de 2 segundos (carregamento tardio e cache de 7 dias).

US-05: Buscar cupcake ✅ · Alta · 3 pts

Ação: Como um cliente, eu quero buscar cupcakes pelo nome ou ingrediente, para que eu encontre rapidamente o produto.

- CA#1: A busca retorna resultados enquanto o cliente digita. CA#2: Mostra "Produto não encontrado" quando não há resultados.
- RN#1: A busca não diferencia maiúsculas de minúsculas.
- RNF#1: Resposta em até 1 segundo.

US-06: Visualizar detalhes do produto ✅ · Alta · 3 pts

Ação: Como um cliente, eu quero ver foto, descrição, ingredientes e preço de um cupcake, para decidir se compro.

- CA#1: Foto ampliada, descrição, ingredientes e preço. CA#2: Botão "Adicionar ao carrinho" com seletor de quantidade.
- RN#1: O preço exibido já inclui impostos ("Impostos inclusos").

Revisão PIT II: aviso "Últimas N unidades" quando o estoque está baixo.

US-07: Adicionar ao carrinho ✅ · Alta · 3 pts

Ação: Como um cliente, eu quero adicionar cupcakes ao carrinho com a quantidade desejada, para comprar vários itens num único pedido.

- CA#1: Quantidade por item. CA#2: Subtotal atualizado na hora. CA#3: Remover itens.
- RN#1: Máximo de 20 unidades por item (e nunca mais do que o estoque).
- RN#2: O carrinho continua salvo mesmo se o app for fechado.
- RN#3 *(novo)*: Os preços do carrinho são sempre recalculados pelo servidor.
- RN#4 *(novo)*: O carrinho é de cada usuário: outra pessoa que entrar no mesmo aparelho não vê os itens do anterior (BUG-09).

Pedido e pagamento

US-08: Realizar pedido 🟡 · Alta · 8 pts

Ação: Como um cliente com itens no carrinho, eu quero confirmar o pedido com o endereço de entrega, para que os cupcakes sejam preparados e entregues.

- CA#1: O cliente informa o CEP, e rua, bairro e cidade são preenchidos automaticamente.
- CA#2: O sistema calcula o frete. CA#3: Um resumo do pedido aparece antes da confirmação.
- RN#1: Endereço validado pela API de CEP (ViaCEP).
- RN#2: Pedido mínimo de **R$ 15,00 em produtos** (o frete não conta).
- RN#3 *(novo)*: Frete por zona: mesma cidade da loja R$ 7,00 / 45 min; outras cidades do estado R$ 15,00 / 120 min; fora do estado não há entrega.
- RNF#1: Cálculo do frete em até 3 segundos (tempo limite de 2 s na consulta de CEP).

Revisão PIT II: a opção "selecionar endereço cadastrado" (CA#1 do PIT I) fica para a Sprint 4. Hoje o cliente informa o endereço a cada pedido.

US-09: Escolher forma de pagamento 🟡 · Alta · 5 pts

Ação: Como um cliente finalizando o pedido, eu quero escolher entre cartão, PIX ou dinheiro, para que meu pedido seja processado.

- CA#1: Opções de cartão, PIX e dinheiro na entrega. CA#2: QR Code gerado para o PIX. CA#3: Cartão processado por gateway.
- RN#1: Cartão em até 3x sem juros. RN#2: O PIX expira em 30 minutos; depois disso o pedido é cancelado e o estoque volta.

- RN#3 *(novo)*: Só os 4 últimos dígitos do cartão são guardados.
- RNF#1: Comunicação por HTTPS/TLS.

Revisão PIT II: o **gateway é simulado**: cartão válido é aprovado, cartão terminado em 0002 é recusado e o PIX tem o botão "Simular pagamento". Ligar um gateway real (Mercado Pago, Stripe) exige só trocar a implementação de `PagamentoService`.

US-10: Acompanhar status do pedido 🟡 · Alta · 5 pts

Ação: Como um cliente que fez um pedido, eu quero acompanhar o status em tempo real (em preparação, saiu para entrega, entregue), para saber quando chega.

- CA#1: Status exibido em tempo real, numa linha do tempo.
- CA#2: Uma notificação é gerada a cada mudança de status.
- RN#1: O tempo estimado de entrega depende da zona de entrega (ver US-08 RN#3).
- RNF#1: A mudança aparece para o cliente em até 10 segundos.

Revisão PIT II: a notificação **push** virou **notificação dentro do app** (sino com contador), e a tela do pedido consulta o status a cada 10 s. O RNF original pedia 5 s; o prazo foi ajustado para reduzir a carga no servidor gratuito.

US-19: Cancelar pedido (cliente) ✅ · Média · 2 pts · *nova no PIT II*

Ação: Como um cliente, eu quero cancelar um pedido que ainda não saiu para entrega, para desistir da compra sem ligar para a loja.

- CA#1: O botão "Cancelar pedido" aparece só em "Aguardando pagamento" e "Em preparação".
- RN#1: O cancelamento devolve o estoque.

Origem: o método `cancelarPedido()` do diagrama de classes do PIT I não tinha história de usuário correspondente.

Administração

US-14: Gerenciar produtos ✅ · Alta · 8 pts

Ação: Como administrador, eu quero adicionar, editar e remover cupcakes com foto, descrição, ingredientes, preço e categoria, para manter o cardápio atualizado.

- CA#1: Upload da foto. CA#2: Campos obrigatórios validados antes de salvar.
- RN#1: Cupcakes em pedidos em andamento não podem ser removidos.
- RN#2 *(novo)*: A remoção é lógica (o cupcake sai do cardápio, mas continua nos pedidos antigos).
- RNF#1: Upload aceita JPG e PNG de até 5MB (formato conferido pelo conteúdo do arquivo).

US-15: Gerenciar estoque ✅ · Alta · 5 pts

Ação: Como administrador, eu quero atualizar a quantidade disponível de cada cupcake, para que os clientes vejam a disponibilidade real.

- CA#1: Alerta no painel quando o estoque está abaixo de 5 unidades. CA#2: Estoque zero aparece como "Indisponível".

- RN#1: O estoque baixa automaticamente a cada pedido confirmado e volta se o pedido for cancelado.

US-16: Gerenciar pedidos ✅ · Alta · 5 pts

Ação: Como administrador, eu quero ver todos os pedidos e atualizar seus status, para gerenciar preparo e entrega.

- CA#1: Filtro por status e por período. CA#2: Mudar o status notifica o cliente automaticamente.
- RN#1: Pedidos entregues não podem ser cancelados.
- RN#2 *(novo)*: O status só avança na ordem *Em preparação → Saiu para entrega → Entregue*.
- RN#3 *(novo)*: Pagamento em dinheiro é marcado como pago quando o pedido é entregue.

US-20: Gerenciar categorias ✅ · Média · 2 pts · *nova no PIT II*

Ação: Como administrador, eu quero criar e remover categorias, para organizar o cardápio.

- RN#1: O nome da categoria é único. RN#2: Só é possível remover uma categoria sem cupcakes.

Origem: a classe `Categoria` do PIT I não tinha história para mantê-la.

Planejadas para a Sprint 4 ⏳

ID	História	Pontos
US-11	Avaliar pedido (nota de 1 a 5 e comentário, só após a entrega)	3
US-12	Histórico de pedidos com "Repetir pedido" (a listagem já existe)	3
US-13	Favoritos (ícone de coração, lista na conta)	2
US-17	Relatório de vendas por período, com exportação em PDF	8

Requisitos não funcionais gerais

ID	Requisito
RNF-G1	Interface mobile-first, utilizável a partir de 360 px de largura
RNF-G2	Senhas guardadas com hash BCrypt; proteção CSRF em todos os formulários
RNF-G3	Acesso de um cliente ao pedido de outro responde "não encontrado" (404)
RNF-G4	Banco PostgreSQL com o esquema versionado (Flyway)
RNF-G5	Regras de negócio cobertas por testes automatizados (JUnit)
RNF-G6	Erros mostram mensagens amigáveis, sem detalhes técnicos para o usuário

3. Product Backlog e Sprints

Backlog priorizado do PIT I, atualizado com o andamento real do PIT II.

ID	História	Pontos	Prioridade	Sprint	Situação no PIT II
US-01	Cadastrar conta	3	Alta	1	✅ Entregue
US-02	Fazer login	2	Alta	1	✅ Entregue
US-03	Fazer logout	1	Alta	1	✅ Entregue
US-18	Recuperar senha	3	Alta	1	🟡 Entregue (e-mail simulado)
US-04	Navegar pela vitrine	5	Alta	1	✅ Entregue
US-05	Buscar cupcake	3	Alta	2	✅ Entregue
US-06	Visualizar detalhes	3	Alta	2	✅ Entregue
US-07	Adicionar ao carrinho	3	Alta	2	✅ Entregue
US-08	Realizar pedido	8	Alta	2	🟡 Entregue (sem endereços salvos)
US-09	Escolher forma de pagamento	5	Alta	2	🟡 Entregue (gateway simulado)
US-10	Acompanhar status do pedido	5	Alta	3	🟡 Entregue (notificação no app)
US-14	Gerenciar produtos (Admin)	8	Alta	3	✅ Entregue
US-15	Gerenciar estoque (Admin)	5	Alta	3	✅ Entregue
US-16	Gerenciar pedidos (Admin)	5	Alta	3	✅ Entregue
US-19	Cancelar pedido (cliente) *(nova)*	2	Média	3	✅ Entregue
US-20	Gerenciar categorias (Admin) *(nova)*	2	Média	3	✅ Entregue
US-21	Alterar senha logado *(nova, revisão de qualidade)*	2	Média	3	✅ Entregue
US-11	Avaliar pedido	3	Média	4	⏳ Planejada
US-12	Histórico com "Repetir pedido"	3	Média	4	⏳ Parcial (listagem pronta)

ID	História	Pontos	Prioridade	Sprint	Situação no PIT II
US-13	Adicionar aos favoritos	2	Média	4	⏳ Planejada
US-17	Relatórios de vendas (Admin)	8	Média	4	⏳ Planejada

Resumo por sprint

Sprint	Objetivo	Pontos	Situação
1	Acesso e vitrine	14	Concluída
2	Compra: busca, carrinho, pedido e pagamento	22	Concluída
3	Pós-venda e administração	29 (23 + 6 das histórias novas)	Concluída
4	Fidelização e gestão	16	Próximo ciclo

Velocidade: 65 pontos entregues nas Sprints 1 a 3, de 81 pontos no backlog total (80%), com média de cerca de 22 pontos por sprint.

Definição de pronto (Definition of Done)

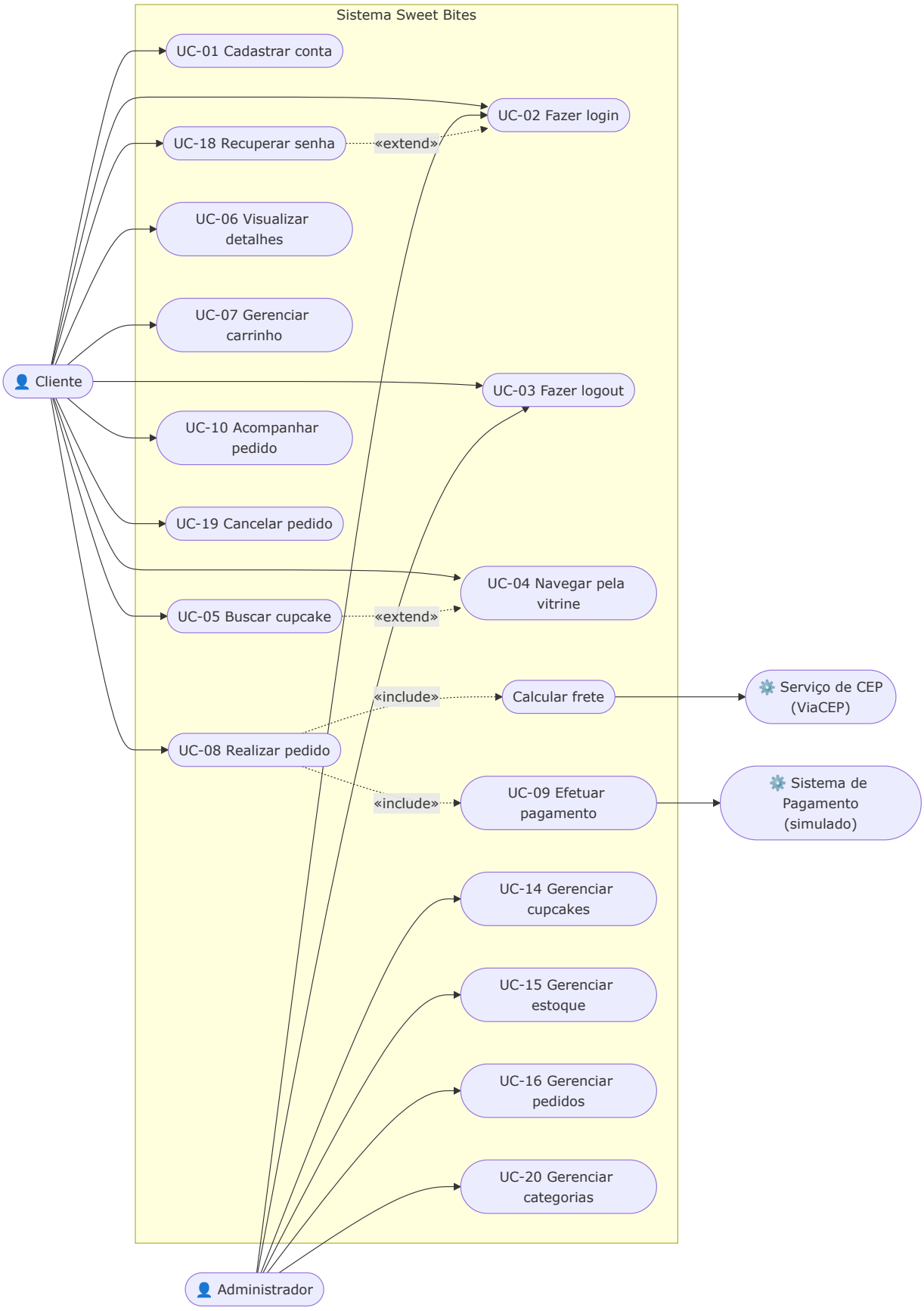
Uma história só é considerada pronta quando:

1. todos os critérios de aceitação estão implementados;
2. as regras de negócio estão no back-end, e não só na tela;
3. existe teste automatizado para as regras principais (`./mvnw test` passa);
4. a história foi verificada manualmente no navegador, com layout de celular;
5. a documentação foi atualizada.

4. Casos de uso (revisados)

4.1 Diagrama de casos de uso geral

O GitHub não desenha diagramas de caso de uso UML nativos, por isso este foi feito com um fluxograma Mermaid que segue a notação: atores fora da fronteira do sistema, casos de uso como elipses, e `<<include>>` / `<<extend>>` nas setas pontilhadas.



Correções feitas em relação ao PIT I:

- O caso "Fazer login" estava **duplicado** e ligado a si mesmo. Agora há um único caso, compartilhado por Cliente e Administrador.

- Faltavam casos que existiam nas histórias: Recuperar senha, Logout, Buscar, Gerenciar pedidos. Também foram incluídos os casos novos (Cancelar pedido, Gerenciar categorias).
- O ator "Serviço de CEP" foi adicionado, porque a RN de validação de endereço depende dele.
- "Efetuar pagamento" passou a ser `<<include>>` de "Realizar pedido", já que não existe pagamento fora de um pedido.
- Os rótulos em inglês ("Customer", "Payment System") foram traduzidos.
- "Visualizar relatórios" saiu deste diagrama, por ser da Sprint 4.

4.2 Caso de uso expandido: UC-08 Realizar pedido (revisado)

Campo	Descrição
Identificador	UC-08
Atores	Cliente (principal), Serviço de CEP, Sistema de Pagamento
Pré-condições	Cliente autenticado e com ao menos um item no carrinho
Pós-condições	Pedido registrado, estoque reservado e cliente notificado. Status: "Em preparação" (cartão/dinheiro) ou "Aguardando pagamento" (PIX)

Fluxo principal

1. O cliente abre o carrinho e toca em **Finalizar pedido**.
2. O sistema exibe a tela de checkout com o resumo dos itens (preços recalculados no servidor).
3. O cliente informa o **CEP**.
4. O sistema consulta o CEP, preenche rua, bairro e cidade, e calcula **frete e tempo estimado** (RN: frete por zona).
5. O cliente informa número e complemento.
6. O cliente escolhe a **forma de pagamento**: cartão, PIX ou dinheiro.
7. O cliente toca em **Confirmar pedido**.
8. O sistema valida de novo, no servidor: estoque (com bloqueio da linha no banco), máximo de 20 por item, pedido mínimo de R$ 15,00 e frete.
9. O sistema processa o pagamento.
10. O sistema grava pedido, itens, pagamento e entrega numa **única transação** e baixa o estoque.
11. O sistema exibe a **tela de confirmação** com o número do pedido e a previsão de entrega, e esvazia o carrinho.
12. O sistema gera a notificação "Pedido confirmado".

Curso alternativo Alfa: CEP inválido ou fora da área

- A4. O CEP não existe ou fica fora do estado da loja.
- A5. O sistema mostra "CEP não encontrado. Verifique o endereço." ou "Ainda não entregamos em ".
- A6. O caso de uso volta ao passo 3.

Curso alternativo Beta: pagamento recusado

- B9. O gateway recusa o cartão.
- B10. O sistema mostra "Pagamento não aprovado. Tente outro método de pagamento." **Nada é gravado e o estoque não é alterado.** O endereço continua preenchido.
- B11. O caso de uso volta ao passo 6.

Curso alternativo Gama: estoque ou valor insuficiente *(novo)*

- G8. Outro cliente comprou as últimas unidades, ou o subtotal ficou abaixo de R$ 15,00.
- G9. O sistema mostra a mensagem específica (ex.: "Red Velvet Morango: só temos 2 unidade(s) em estoque.").
- G10. O cliente volta ao carrinho para ajustar.

Fluxo PIX *(detalhado)*: no passo 9 o sistema gera o QR Code e o código "copia e cola", válidos por 30 minutos, e o pedido fica "Aguardando pagamento". Quando o pagamento é confirmado, o pedido passa para "Em preparação". Se o PIX vence sem pagamento, um job cancela o pedido e devolve o estoque.

4.3 Caso de uso expandido: UC-01 Cadastrar conta (revisado)

Campo	Descrição
Identificador	UC-01
Ator	Cliente
Pré-condições	Não existe conta com o mesmo e-mail
Pós-condições	Conta criada com o perfil CLIENTE; e-mail de boas-vindas enviado (simulado)

Fluxo principal

1. O usuário toca em **Cadastre-se** na tela de login.
2. O sistema exibe o formulário: Nome, E-mail, Telefone, Senha e Confirmar senha.
3. O usuário preenche e toca em **Criar conta**.
4. O sistema valida os campos (RN01: e-mail único; RN02: senha com 8 caracteres ou mais, com letras e números; telefone com DDD; confirmação igual à senha).
5. O sistema grava a conta com a senha criptografada (BCrypt).
6. O sistema envia o e-mail de boas-vindas.
7. O sistema volta para o login com a mensagem "Conta criada com sucesso!".

Curso alternativo Alfa: dados inválidos

- A4. Um ou mais campos são inválidos.
- A5. O sistema destaca os campos em vermelho e mostra **uma** mensagem embaixo de cada um.
- A6. O caso de uso volta ao passo 3, mantendo os dados já digitados (exceto as senhas).

4.4 Caso de uso expandido: UC-16 Gerenciar pedidos *(novo)*

Campo	Descrição
Ator	Administrador
Pré-condições	Administrador autenticado
Pós-condições	Status do pedido alterado e cliente notificado

Fluxo principal

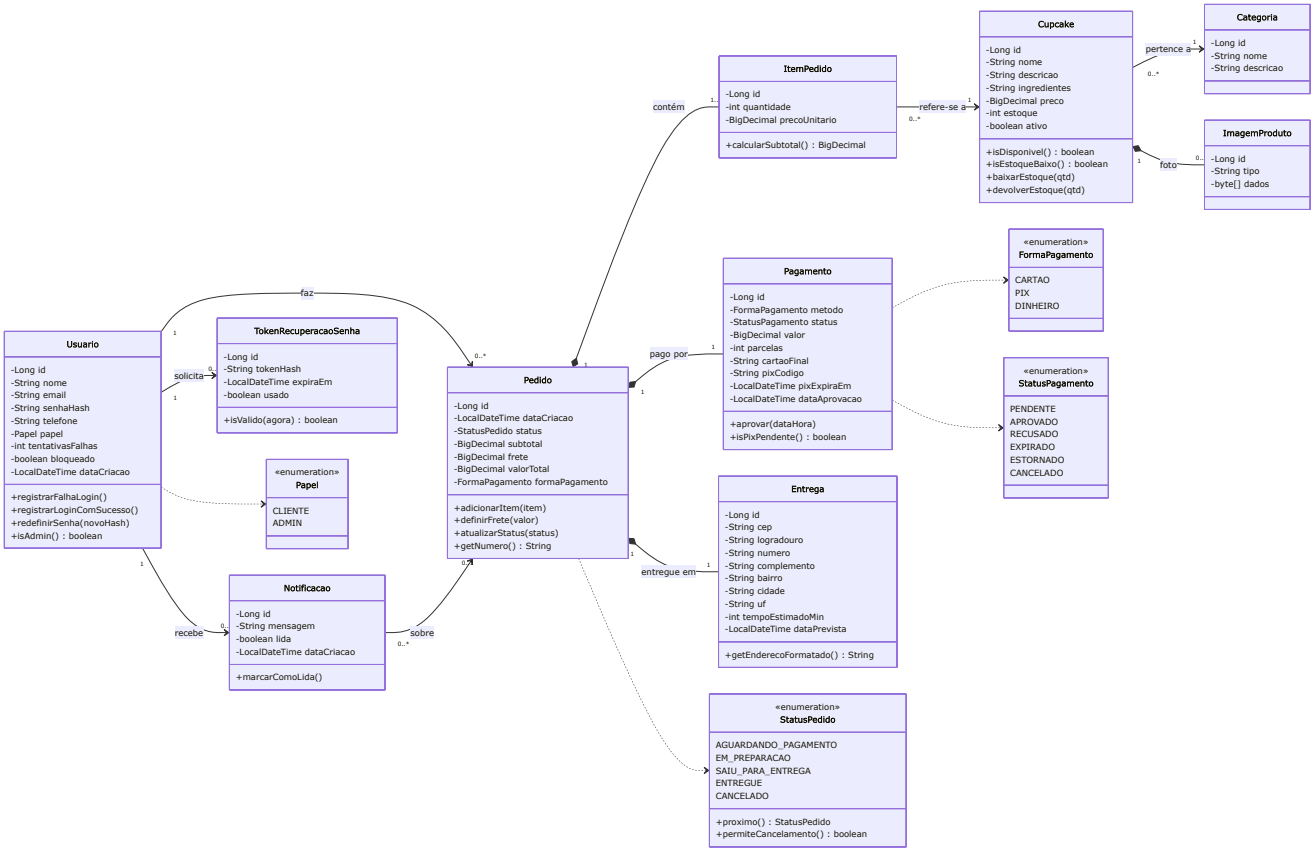
1. O administrador abre **Pedidos** e, se quiser, filtra por status e período.
2. O administrador abre um pedido.
3. O sistema mostra itens, cliente, endereço, pagamento e o botão do próximo status permitido.

4. O administrador toca em **Marcar como:** .
5. O sistema atualiza o status e notifica o cliente. Se o pedido em dinheiro for entregue, marca o pagamento como pago.

Curso alternativo: cancelar. O administrador toca em **Cancelar pedido** e confirma. O sistema devolve o estoque e estorna o pagamento aprovado. Pedidos **entregues** não exibem essa opção, e o servidor também recusa o cancelamento.

5. Diagrama de classes (revisado)

Este é o diagrama do modelo de domínio, ou seja, das classes de persistência que de fato existem no código, em `situacao-2-codificacao/src/main/java/br/com/sweetbites/model`.



O que foi corrigido em relação ao diagrama do PIT I

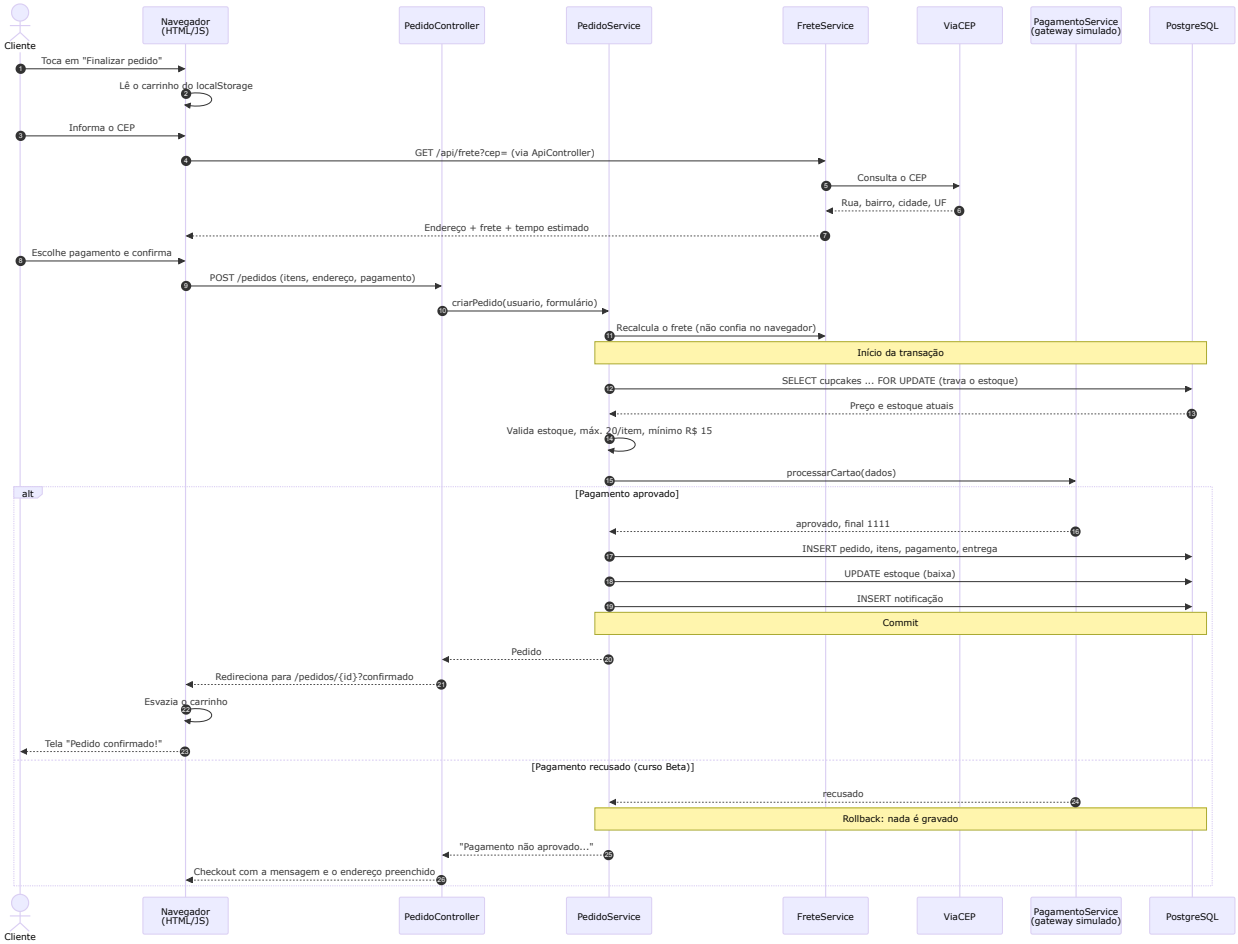
#	Problema no PIT I	Correção
1	A classe **Pagamento aparecia duas vezes**, e uma delas tinha os atributos de Entrega	Uma única classe `Pagamento`, com método, status, valor, parcelas e dados do PIX
2	Atributos de **Entrega ilegíveis** ("endercc Entrega", "valorPrvoant")	`Entrega` com endereço completo (CEP, rua, número, bairro, cidade, UF) e previsão
3	`Pedido` ligado **diretamente** a `Cupcake` (1..1)	A ligação passa por `ItemPedido`: um pedido tem vários itens, e cada item aponta para um cupcake
4	`Categoria` ligada a `ItemPedido`	`Categoria` ligada a `Cupcake`, que é quem de fato tem categoria
5	Composição Usuario ◆ Pedido	Associação simples: o pedido não é "parte" do usuário e é preservado como histórico
6	Valores monetários como `Float`	`BigDecimal`, para não acumular erro de arredondamento em dinheiro
7	`Usuario.endereco` como texto solto	O endereço fica em `Entrega`, uma por pedido; o cliente pode receber em locais diferentes
8	Sem perfil de acesso	`Papel` (CLIENTE/ADMIN) separa o que cada um pode fazer

#	Problema no PIT I	Correção
9	Status como `String` livre	Enumerações (`StatusPedido` , `StatusPagamento` , `FormaPagamento`), com as transições permitidas codificadas
10	Faltavam classes exigidas pelas histórias	`Notificacao` (US-10), `TokenRecuperacaoSenha` (US-18), `ImagemProduto` (US-14), além de `tentativasFalhas` / `bloqueado` (US-02)
11	`Usuario.senha` em texto	`senhaHash` : só o hash BCrypt é guardado

> O diagrama original continua disponível em `imagens/pit1-original/diagrama-de-classes.jpg` , para comparação.

6. Diagramas comportamentais (revisados)

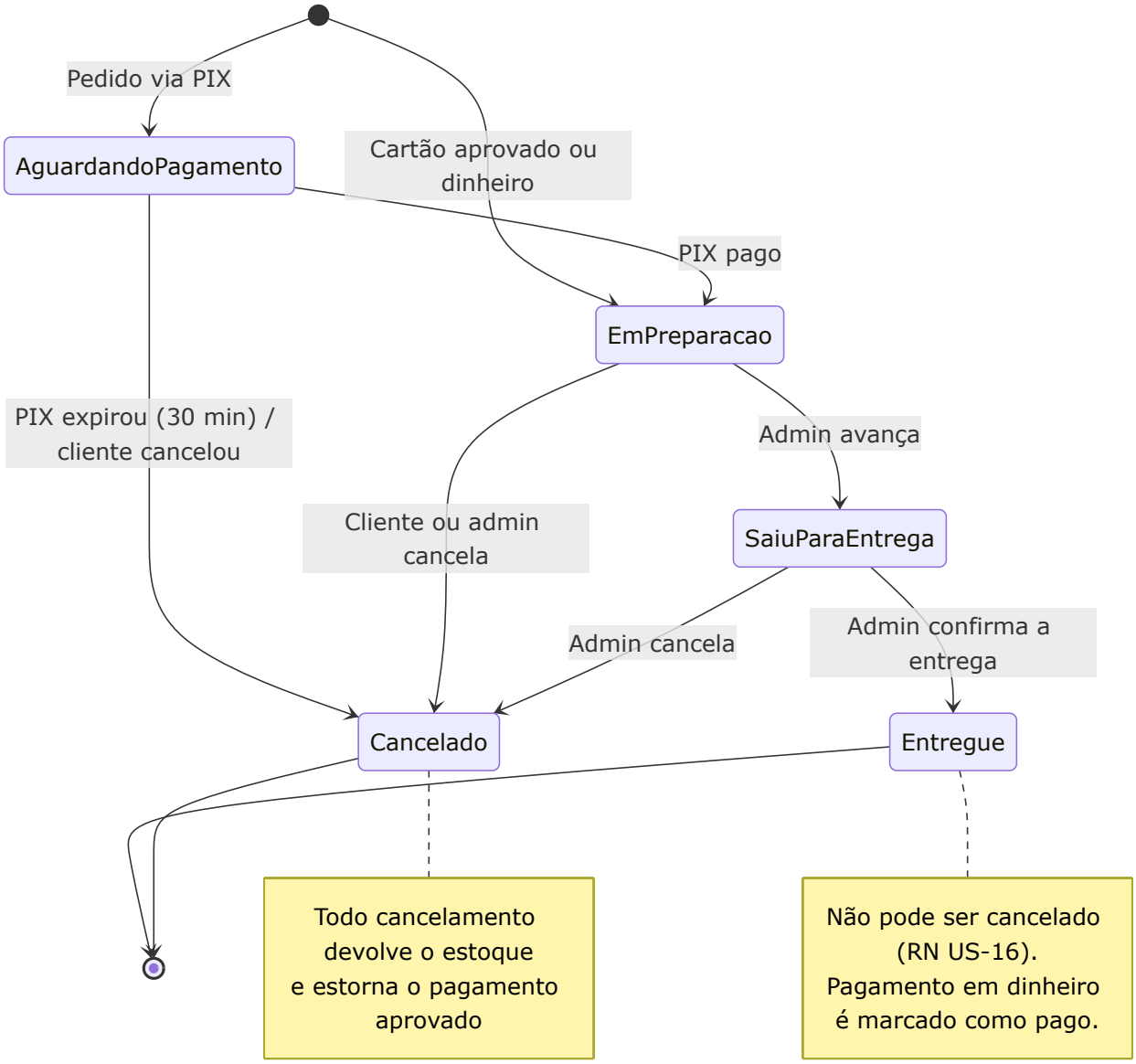
6.1 Diagrama de sequência: Realizar pedido (UC-08)



Correções em relação ao PIT I:

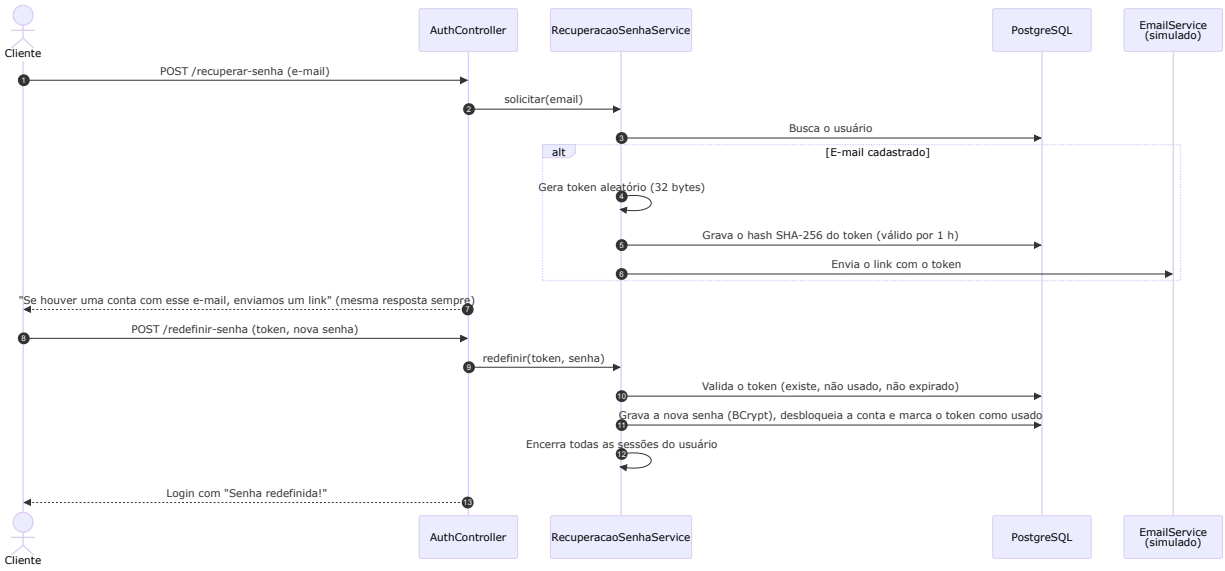
- No diagrama original, o **app enviava os dados do cartão direto ao Sistema de Pagamento**, sem passar pelo servidor. Isso permite alterar valores pelo cliente e expõe credenciais do gateway no aplicativo. Agora **só o servidor** fala com o gateway, e o valor cobrado é calculado no servidor.
- Incluídos a **consulta de CEP**, o **bloqueio de estoque** (evita vender a última unidade para dois clientes) e o fluxo de **recusa com rollback**.
- O título duplicado ("'Realizar Pedido' app - Realizar Pedido") foi corrigido.

6.2 Diagrama de estados: ciclo de vida do pedido *(novo)*



As transições permitidas ficam no próprio enum `StatusPedido` (`proximo()` e `permiteCancelamento()`) e são verificadas no servidor. Por isso, mesmo uma chamada feita fora da tela não consegue, por exemplo, cancelar um pedido entregue.

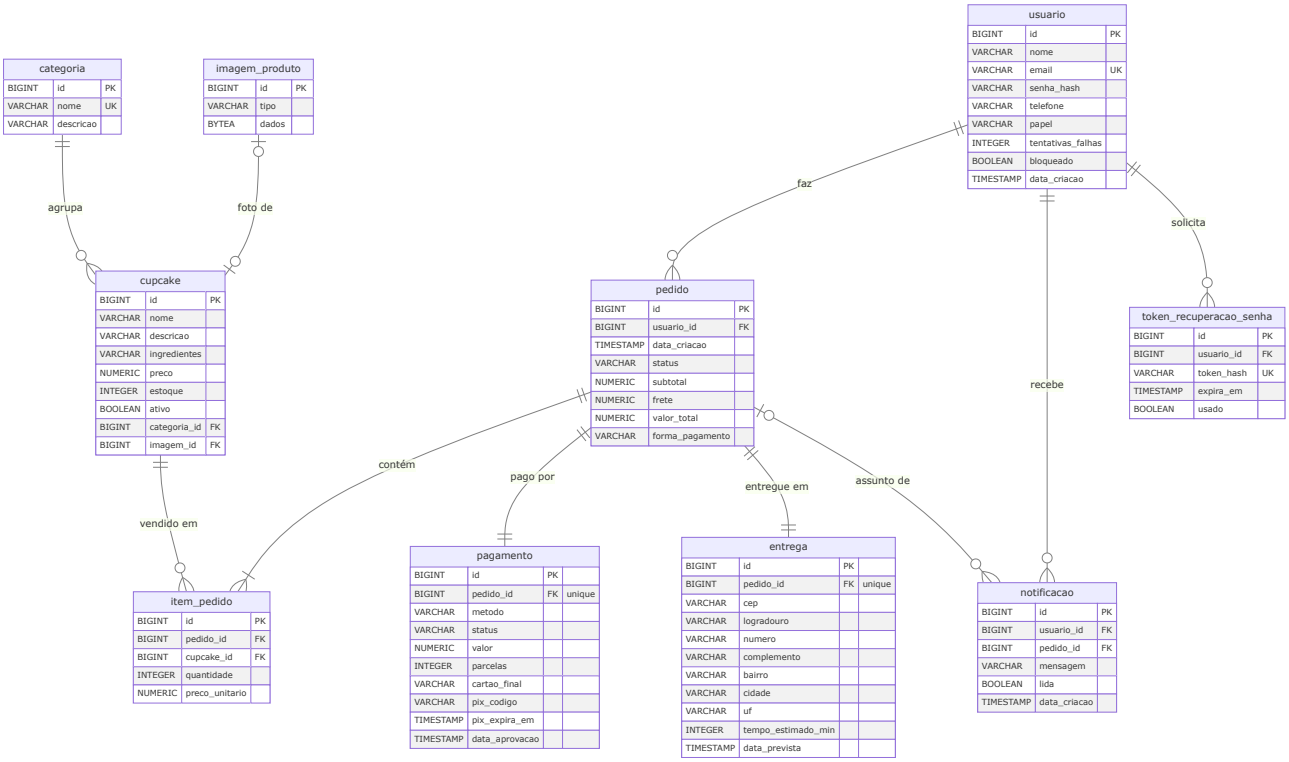
6.3 Diagrama de sequência: Recuperar senha (UC-18) *(novo)*



7. Projeto físico do banco de dados

- **SGBD:** PostgreSQL 16. É popular, gratuito, tem bom suporte em hospedagens e está entre os mais usados do mercado, como o material da disciplina recomenda.
- **Versionamento do esquema:** Flyway. O script `V1__criar_tabelas.sql` é aplicado automaticamente quando a aplicação sobe. Mudanças futuras entram em novos scripts (`V2__...`), nunca editando os já aplicados.
- **Validação:** ao iniciar, o Hibernate confere se as tabelas batem com as entidades Java (`ddl-auto=validate`).

7.1 Diagrama entidade-relacionamento



7.2 Dicionário de dados (tabelas principais)

`usuario`

Coluna	Tipo	Nulo	Regra
id	BIGINT identity	não	Chave primária
nome	VARCHAR(120)	não	
email	VARCHAR(160)	não	**Único**, guardado em minúsculas (RN US-01)
senha_hash	VARCHAR(100)	não	Hash BCrypt, nunca a senha em texto
telefone	VARCHAR(20)	não	Com DDD
papel	VARCHAR(20)	não	`CLIENTE` ou `ADMIN`
tentativas_falhas	INTEGER	não	Padrão 0; ao chegar a 5, bloqueia (RN US-02)

Coluna	Tipo	Nulo	Regra
bloqueado	BOOLEAN	não	Padrão `false`

`cupcake`

Coluna	Tipo	Nulo	Regra
preco	NUMERIC(10,2)	não	`CHECK (preco > 0)`
estoque	INTEGER	não	`CHECK (estoque >= 0)` : o banco impede estoque negativo
ativo	BOOLEAN	não	`false` = removido do cardápio (exclusão lógica, RN US-14)
categoria_id	BIGINT	não	FK → `categoria`
imagem_id	BIGINT	sim	FK → `imagem_produto`

`pedido` / **`item_pedido`**

Coluna	Tipo	Regra
pedido.status	VARCHAR(30)	Valores de `StatusPedido` (ver o diagrama de estados)
pedido.subtotal, frete, valor_total	NUMERIC(10,2)	`valor_total = subtotal + frete`
item_pedido.quantidade	INTEGER	`CHECK (quantidade BETWEEN 1 AND 20)` (RN US-07)
item_pedido.preco_unitario	NUMERIC(10,2)	**Preço do momento da compra**: se o cupcake mudar de preço depois, o histórico não muda

`pagamento`

Coluna	Regra
pedido_id	**Único**: exatamente um pagamento por pedido
cartao_final	Só os 4 últimos dígitos; o número completo e o CVV nunca são gravados
pix_expira_em	Data de criação + 30 min (RN US-09)

`token_recuperacao_senha`

Coluna	Regra
token_hash	SHA-256 do token. Quem tiver acesso ao banco não consegue usar o link
expira_em	Criação + 1 h (RN US-18)
usado	O token vale para uma única redefinição

7.3 Índices

Índice	Motivo
`uk_usuario_email`	Login e checagem de e-mail único
`ix_cupcake_categoria`	Filtro da vitrine por categoria
`ix_pedido_usuario`	Tela "Meus pedidos"

Índice	Motivo
`ix_pedido_status_data`	Filtro do admin por status e período
`ix_item_pedido_pedido`, `ix_item_pedido_cupcake`	Montar o pedido; verificar se o cupcake está em pedido ativo antes de removê-lo
`ix_pagamento_status_metodo`	Job que procura PIX vencido a cada minuto
`ix_notificacao_usuario`	Contador de notificações não lidas no topo de toda tela

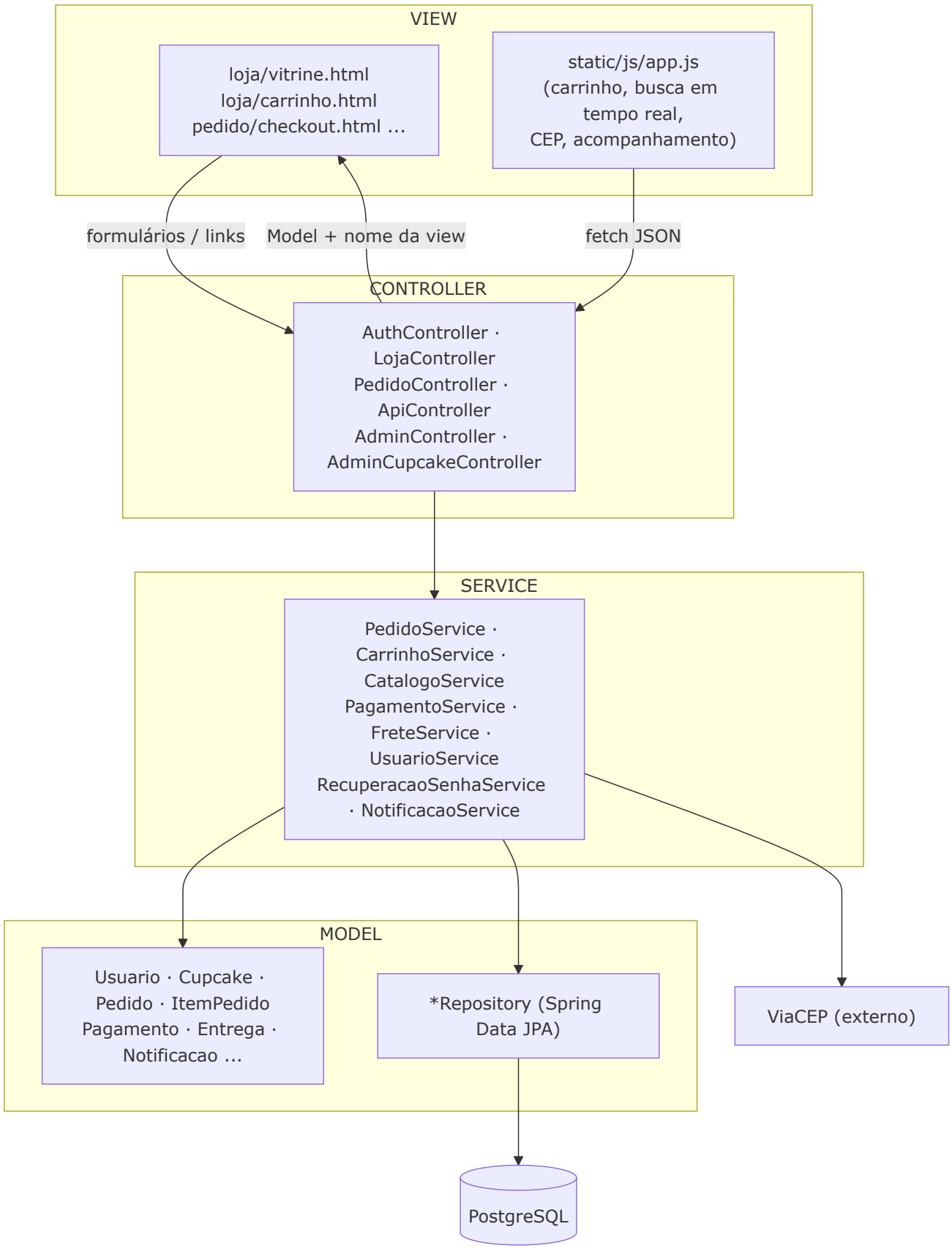
7.4 Por que a imagem fica no banco (BYTEA)?

Hospedagens gratuitas (Render) apagam o disco local a cada novo deploy, então uma foto salva em pasta sumiria. Guardar a imagem no PostgreSQL mantém tudo num lugar só, com backup junto do banco. Como a foto é carregada sob demanda (`LAZY`) e servida com cache de 7 dias, isso não pesa na listagem da vitrine.

8. Arquitetura da solução

8.1 Padrão MVC

O material da disciplina indica o padrão **MVC**, e a aplicação segue esse padrão com Spring MVC. Por boa prática, as regras de negócio ficam numa camada de *services*, separada dos controllers.



Camada	Responsabilidade	Exemplo
View	Apresentação. Não decide regra de negócio	`pedido/checkout.html` mostra o formulário e o resumo
Controller	HTTP: lê parâmetros, chama o service e escolhe a próxima tela	`PedidoController.finalizar()`

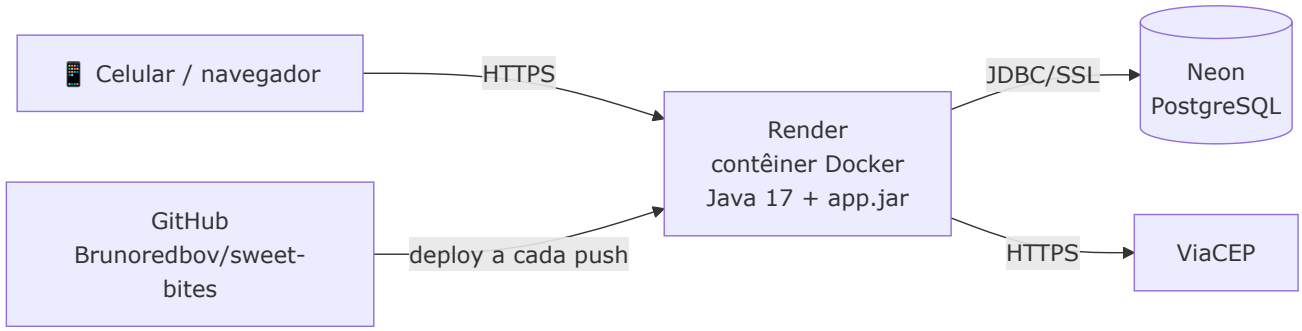
Camada	Responsabilidade	Exemplo
Service	Regras de negócio e transações	`PedidoService.criarPedido()` valida estoque, mínimo e pagamento
Model	Estado e comportamento do domínio	`Cupcake.baixarEstoque()` impede estoque negativo

8.2 Tecnologias

Item	Escolha	Por quê
Linguagem	Java 17 (LTS)	Está na lista do material e tem suporte longo
Framework	Spring Boot 4 (MVC, Security, Data JPA, Validation)	Padrão de mercado para MVC em Java, e é o que os vídeos do material usam
Views	Thymeleaf + HTML5 + CSS3 + JavaScript puro	Exigência do material (HTML/CSS/JS), sem framework pesado no front
Banco	PostgreSQL 16 + Flyway	SGBD popular; o esquema fica versionado no Git
Testes	JUnit 5, Mockito, Spring MockMvc, H2	56 testes rodando em segundos, sem precisar de banco externo
Build	Maven (wrapper `mvnw`)	Quem clona o repositório não precisa instalar o Maven
Deploy	Docker → Render; banco no Neon	Planos gratuitos com URL pública e HTTPS

8.3 Implantação (planejada)

> A publicação será feita no fim do projeto. Até lá, a aplicação roda localmente com `docker compose` (ver o README do código).



8.4 Segurança aplicada

Ameaça	Proteção
Vazamento de senhas	BCrypt; o token de recuperação é guardado como hash SHA-256
Força bruta no login	Bloqueio após 5 tentativas erradas
Formulário enviado por outro site (CSRF)	Token CSRF em todos os formulários (Spring Security)
Injeção de script (XSS)	Thymeleaf escapa o HTML por padrão, e a Content-Security-Policy proíbe scripts inline

Ameaça	Proteção
Cliente vendo pedido de outro	Checagem de dono em toda consulta. Resposta 404 (não 403), para não revelar que o pedido existe
Alteração de preço pelo navegador	O servidor recalcula itens, preços, frete e total
Venda acima do estoque (concorrência)	`SELECT ... FOR UPDATE` no estoque dentro da transação
Upload malicioso	Só JPG/PNG de até 5MB, com o formato conferido pelos bytes do arquivo
Dados de cartão	Nunca gravados, só os 4 últimos dígitos
Senha do admin no código	O admin é criado a partir de variáveis de ambiente (`ADMIN_EMAIL` / `ADMIN_SENHA`)

8.5 Integrações simuladas

Integração	Situação	Como trocar pela real
CEP	**Real** (ViaCEP)	—
Gateway de cartão	Simulada (`PagamentoService`)	Implementar a chamada ao Mercado Pago ou Stripe no mesmo service
PIX	Simulado (QR fictício + botão "Simular pagamento")	Usar a API PIX do gateway, com confirmação por webhook
E-mail	Simulado (`EmailSimuladoService` grava no log)	Criar uma nova implementação da interface `EmailService` (SMTP ou Resend)
Push	Notificação no app + atualização a cada 10 s	Firebase Cloud Messaging

O **modo demonstração** (`MODO_DEMONSTRACAO=true`) mostra na tela os atalhos de teste: o link de redefinição de senha e o botão de pagar o PIX. Desligado, esses atalhos somem e o endpoint de simulação responde 404.

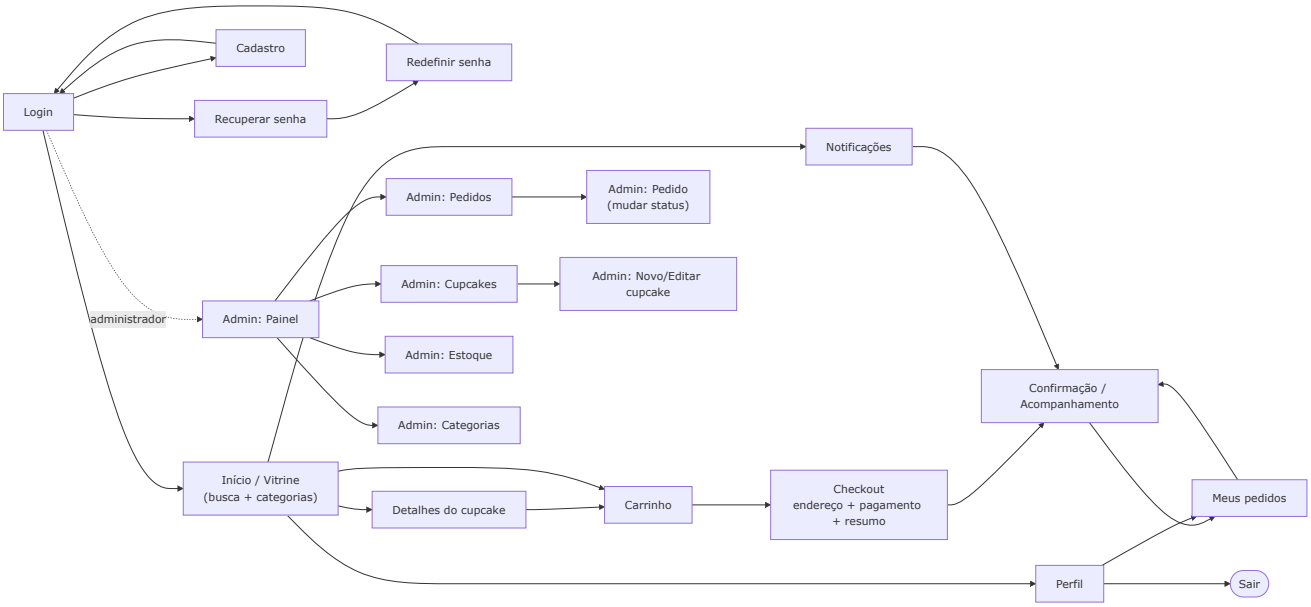
9. Protótipos e telas implementadas

9.1 Do protótipo à tela real

A identidade visual dos mockups do PIT I foi mantida: rosa `#FF6B8A` e branco, cards com foto, botões arredondados e navegação inferior. Algumas mudanças surgiram na implementação:

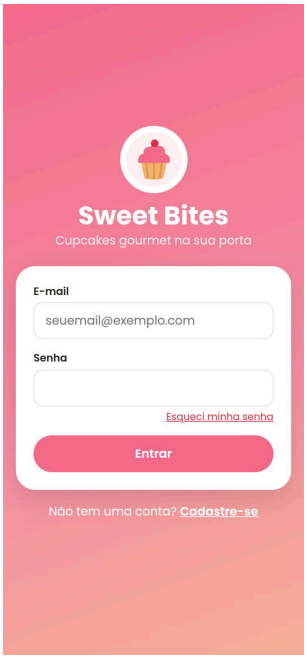
Protótipo PIT I	Implementação PIT II	Motivo
Login social (Google/Facebook)	Removido	Fora das histórias de usuário; exigiria contas de desenvolvedor
Ícone de favorito (coração) nos cards	Removido por enquanto	US-13 é da Sprint 4
"Promo code" no carrinho	Removido	Não havia história de usuário
Avaliações (estrelas) na vitrine	Removido por enquanto	US-11 é da Sprint 4
Navegação: Home, Favoritos, Cardápio, Carrinho, Perfil	Início, Pedidos, Carrinho, Perfil (+ Admin para administradores)	Pedidos ganhou atalho, já que acompanhar a entrega é central (US-10)
Fotos reais de cupcakes	Ilustrações vetoriais no cardápio de exemplo	O admin envia as fotos reais pelo painel (US-14)
—	Sino de notificações com contador	Substitui o push (US-10)
—	Linha do tempo do pedido	Deixa claro o status em tempo real (US-10)

9.2 Mapa navegacional (revisado)



9.3 Telas do cliente (celular, 375 px)

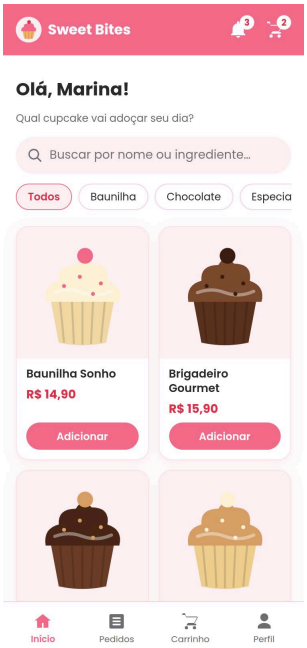
Login



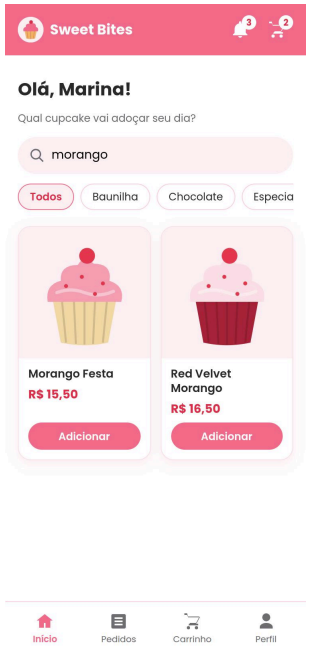
Cadastro (validação)



Vitrine



Busca "morango"



Item indisponível



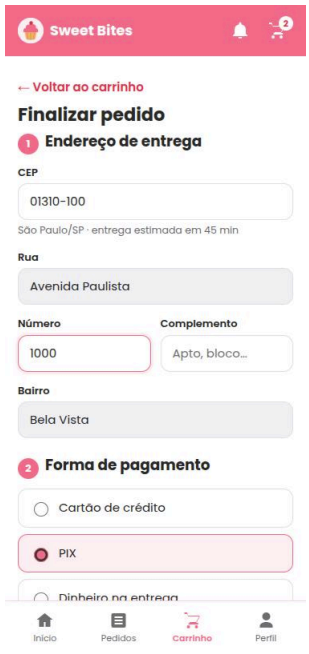
Detalhes



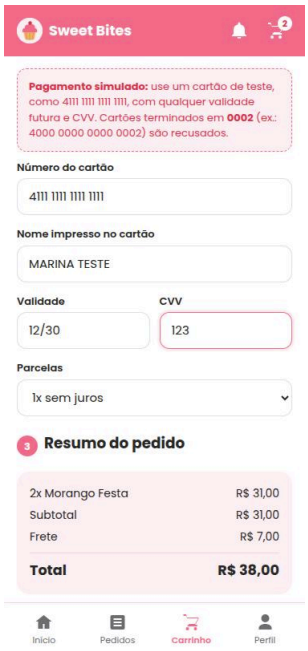
Carrinho



Checkout: endereço



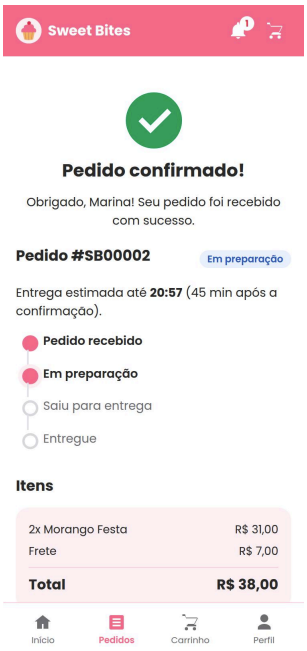
Checkout: pagamento



Pagamento via PIX



Confirmação



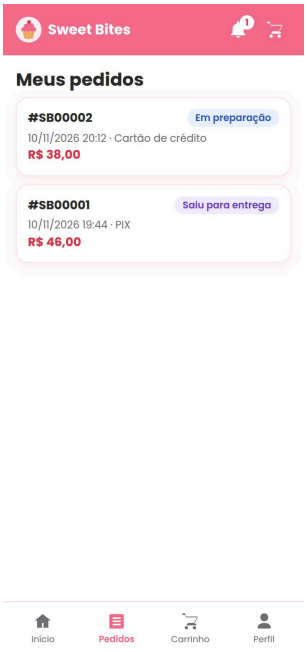
Acompanhamento



Notificações

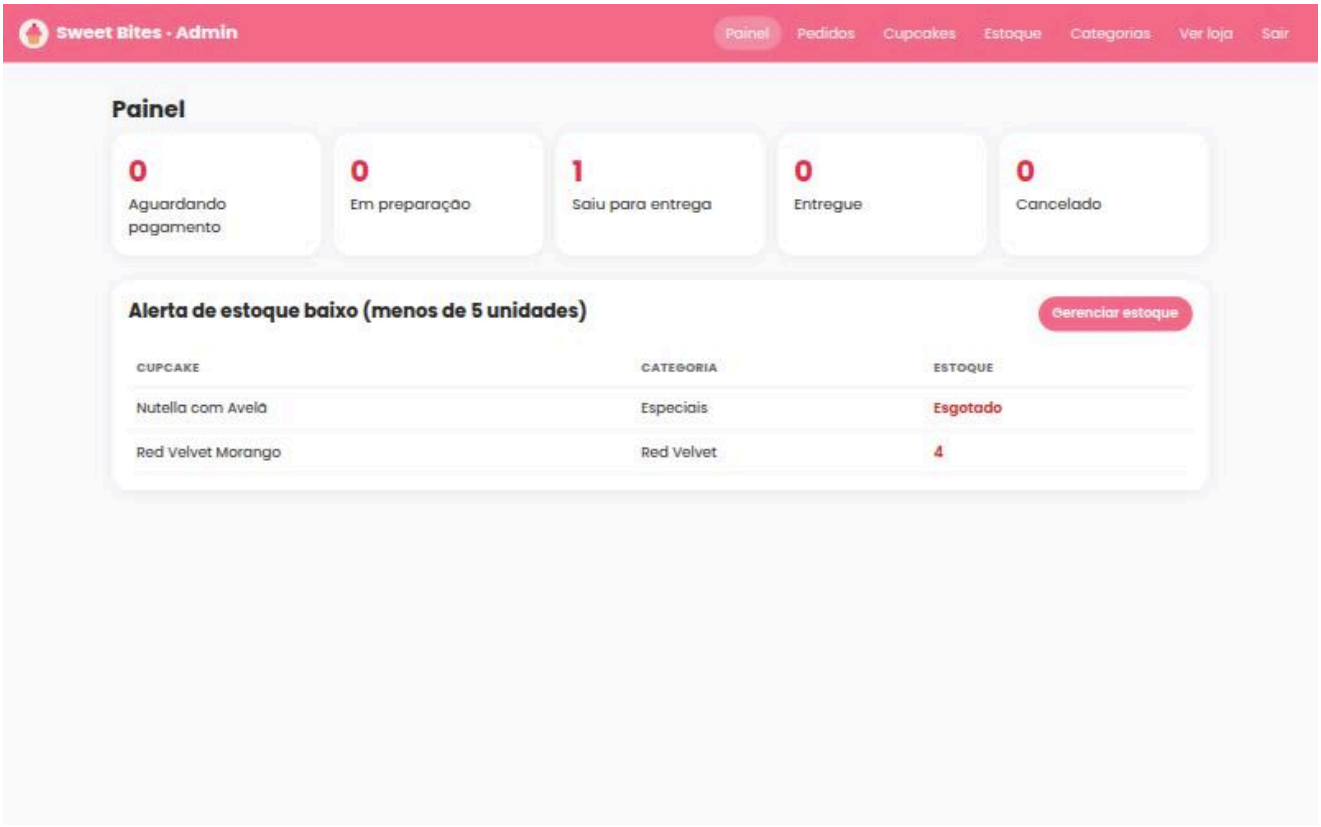


Meus pedidos

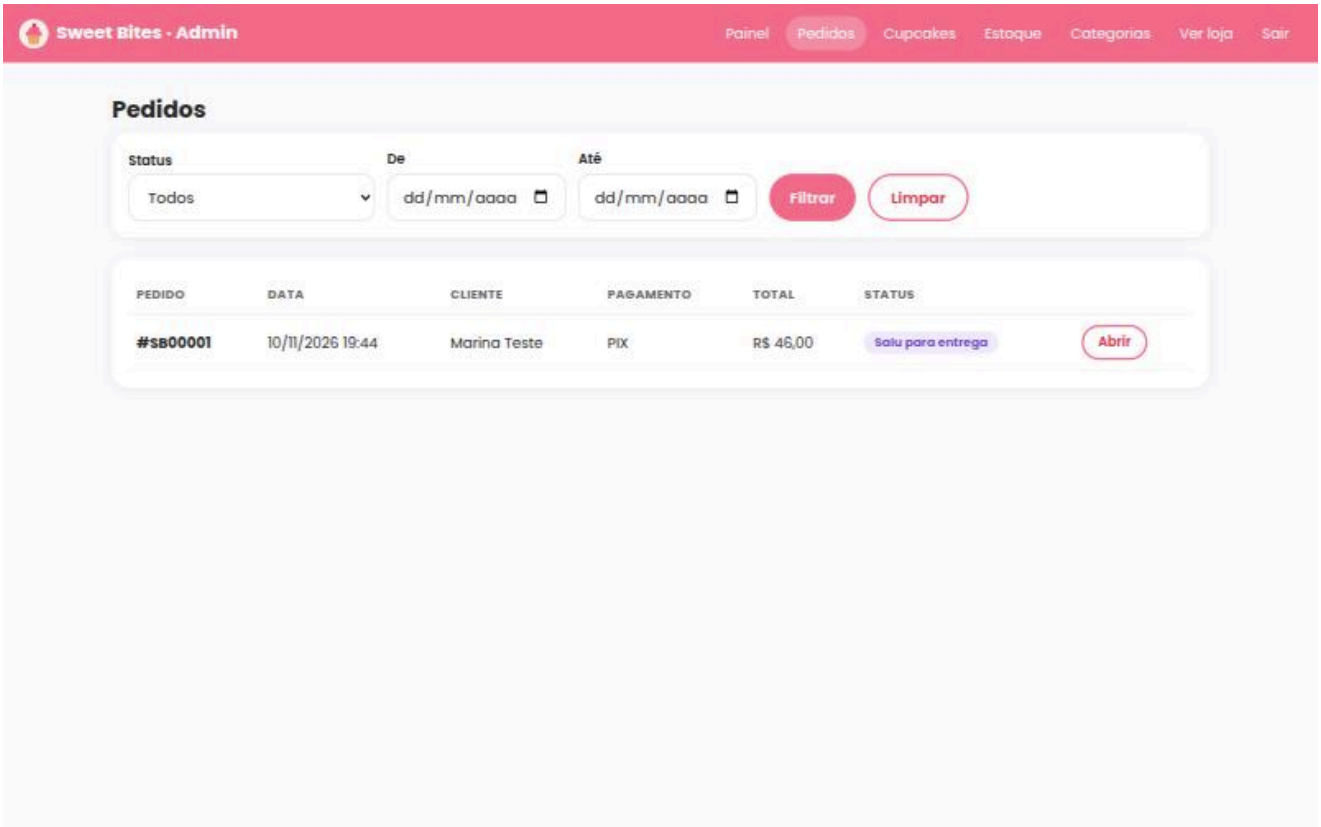


9.4 Telas do administrador (desktop)

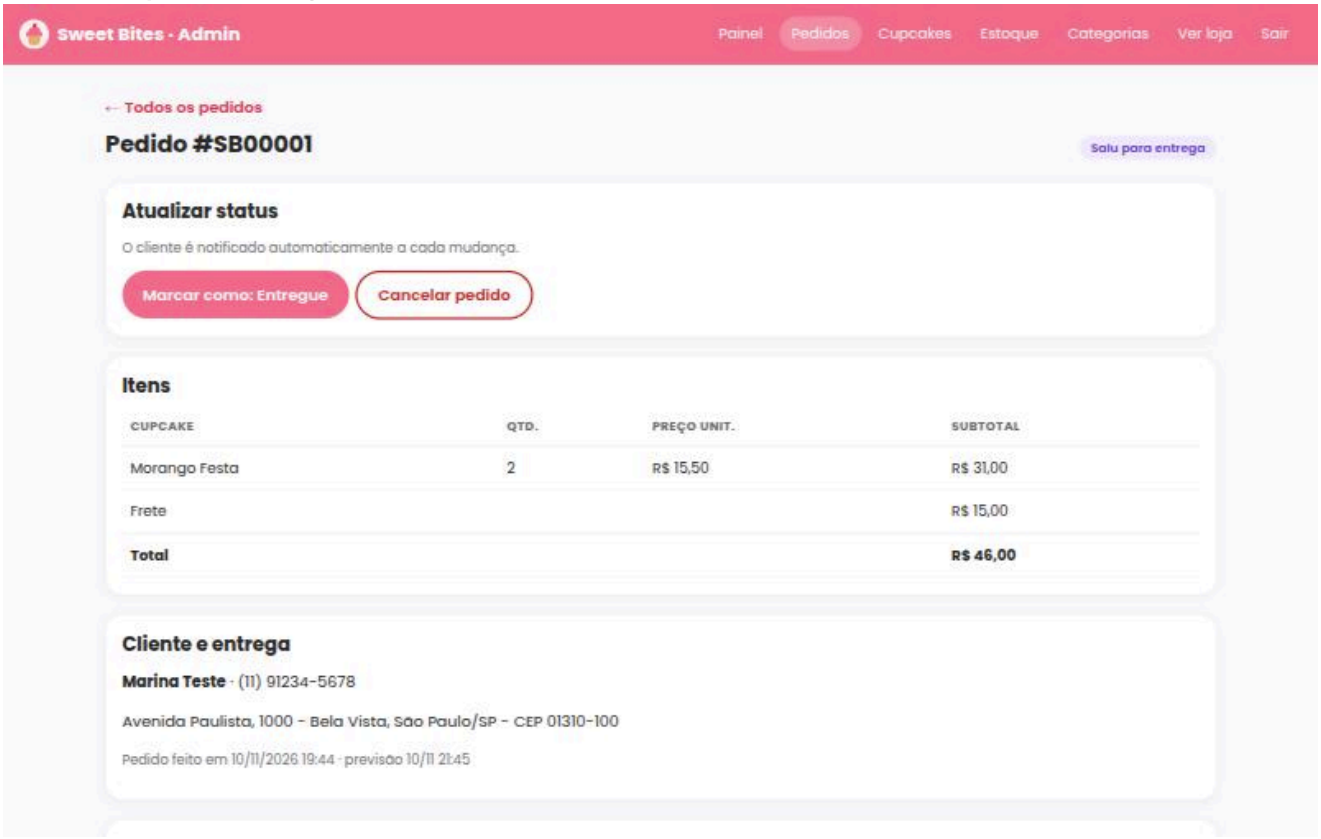
Painel: contadores por status e alerta de estoque baixo



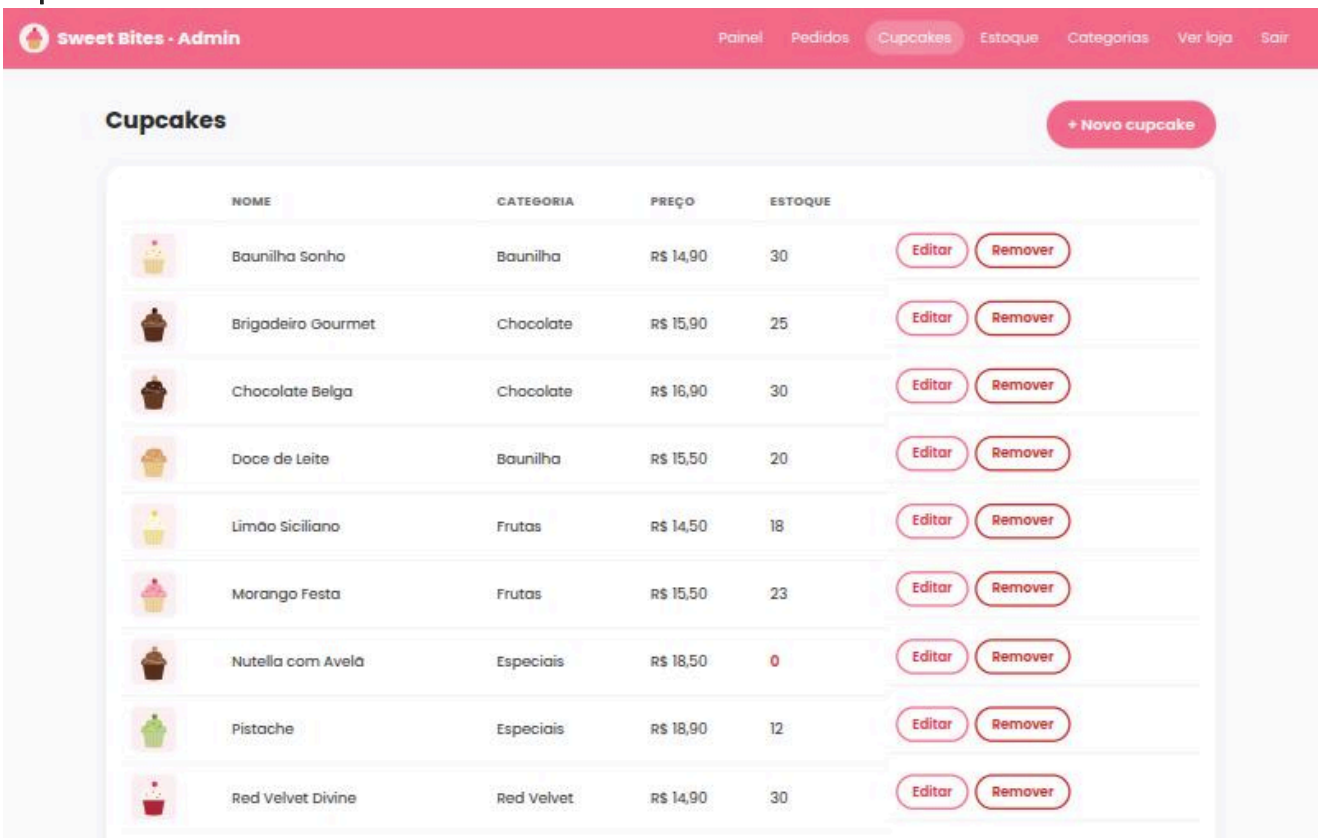
Pedidos: filtro por status e período



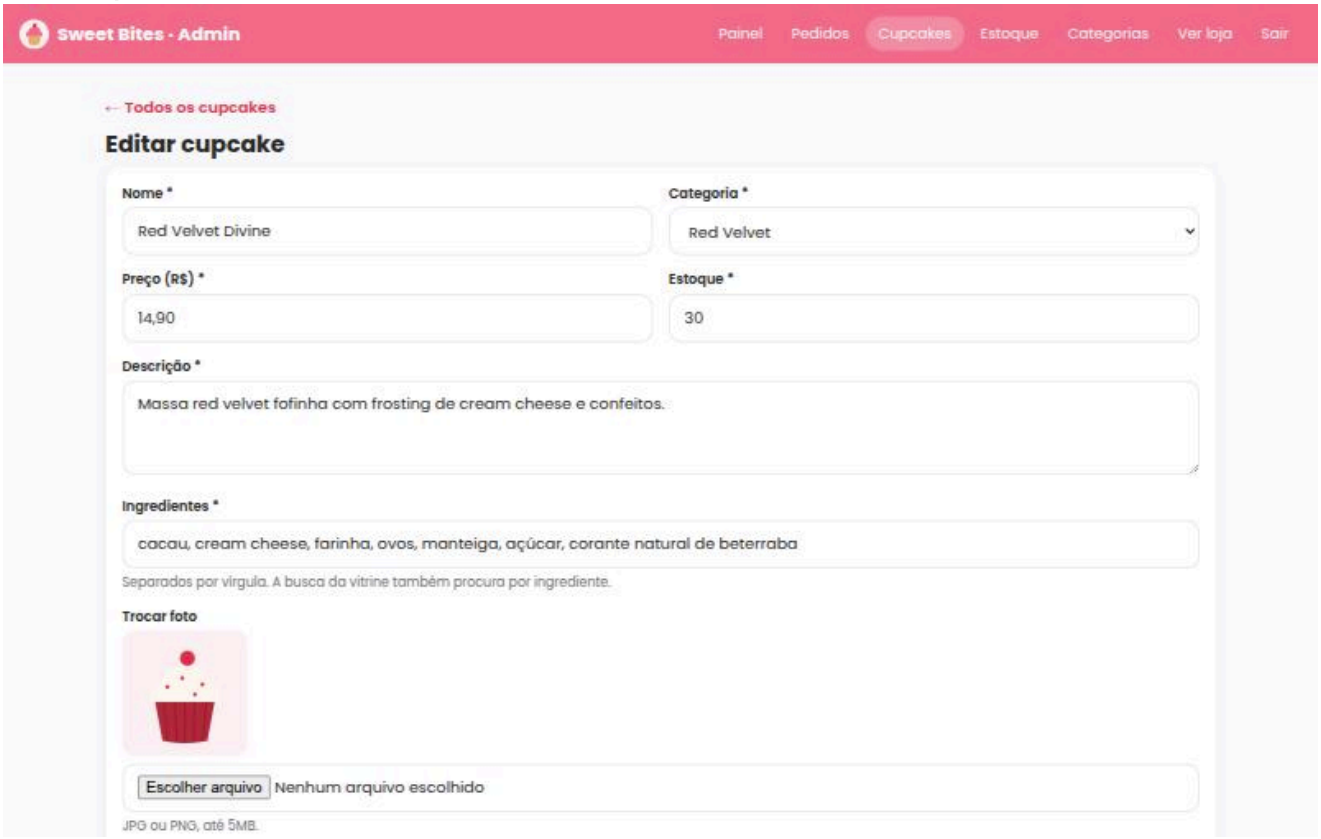
Detalhe do pedido: avançar status ou cancelar



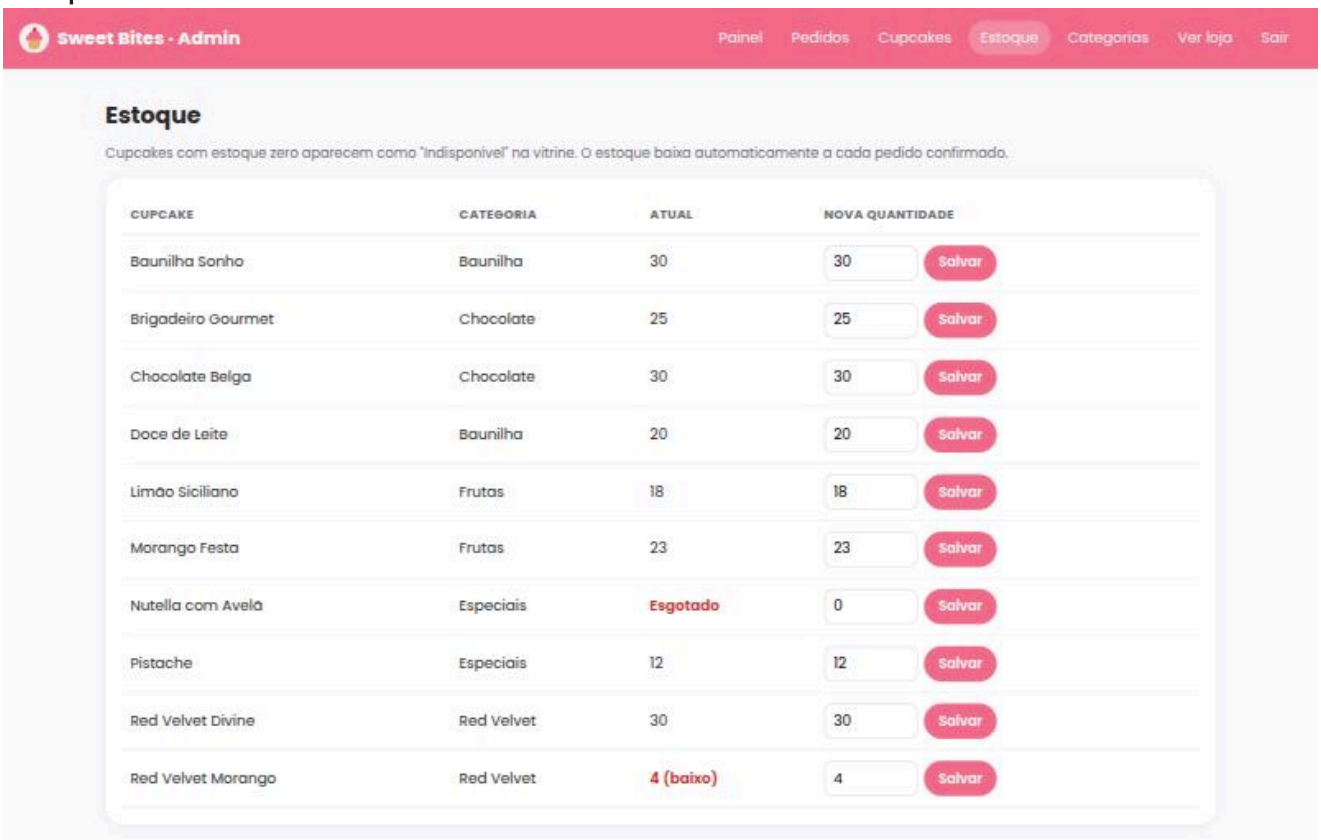
Cupcakes



Editar cupcake: com upload de foto



Estoque



9.5 Artefatos originais do PIT I

Mantidos para comparação em `imagens/pit1-original/` : wireframes, mapa navegacional e mockups.

10. Registro de revisões (melhoria contínua)

Mudanças feitas na documentação do PIT I durante o PIT II e o motivo de cada uma.

#	Artefato	Mudança	Motivo
R01	Plataforma	App nativo → web app mobile-first	O material pede HTML/CSS/JS e um link público para os testes
R02	Histórias	Esclarecido que o pedido mínimo é sobre o valor dos produtos	A regra "pedido mínimo de R$ 15,00" era ambígua quanto ao frete
R03	Histórias	Criada a regra de frete por zona (cidade/estado), com valores configuráveis	"Calcular o frete" não tinha regra definida
R04	Histórias	Pagamento, e-mail e push marcados como simulados	Serviços pagos fora do alcance do projeto acadêmico
R05	Histórias	Novas US-19 (cancelar pedido) e US-20 (gerenciar categorias)	Métodos e classes do diagrama sem história correspondente
R06	Histórias	Novas regras: desbloqueio por redefinição de senha, resposta igual para e-mail inexistente, transições de status do pedido	Lacunas encontradas durante a implementação
R07	Casos de uso	"Fazer login" deixou de estar duplicado; entraram os casos que faltavam; rótulos traduzidos	Erros de modelagem no diagrama original
R08	UC-08	Novo curso alternativo Gama (estoque/valor insuficiente) e detalhamento do PIX	Situações reais que o fluxo original não cobria
R09	Classes	Correções 1 a 11 (ver diagrama de classes)	Classe duplicada, atributos ilegíveis, relacionamentos e tipos incorretos
R10	Sequência	Pagamento passa pelo servidor, não mais direto do app ao gateway	Falha de segurança no desenho original
R11	Novos artefatos	Diagrama de estados do pedido, sequência de recuperação de senha, DER, dicionário de dados, arquitetura	Pedidos do material teórico (projeto físico de BD, MVC)
R12	Protótipos	Removidos elementos sem história (login social, cupom)	Manter o protótipo alinhado ao backlog
R13	Histórias	Nova US-21 (alterar senha logado); novas regras: carrinho por usuário e aviso de sessão expirada	Revisão de qualidade pós-aceitação (BUG-06 a BUG-14 do laudo)

Feedback dos colegas sobre a documentação

O material pede que os colegas critiquem a solução. Registre aqui as sugestões recebidas e o que foi feito com cada uma:

Colega	Sugestão	Aceita?	O que foi feito

Parte 2: Codificação e manual de uso (Situação-Problema 2)

Sweet Bites: código da aplicação (Situação-Problema 2)

Loja virtual de cupcakes gourmet, com vitrine, carrinho, pedido, pagamento e acompanhamento da entrega, além de um painel do administrador. Foi construída a partir da especificação do PIT I (histórias de usuário, UML e protótipos).

Tecnologias

Camada	Tecnologia
Back-end	Java 17 + Spring Boot 4 (Spring MVC, Spring Security, Spring Data JPA)
Padrão de projeto	MVC: `model` (entidades), `controller`, `templates` (views); regras de negócio em `service`
Front-end	HTML5 + CSS3 + JavaScript puro, renderizados com Thymeleaf, layout mobile-first
Banco de dados	PostgreSQL 16, com migrations versionadas pelo Flyway
Testes	JUnit 5 + Mockito + Spring MockMvc (56 testes)
Hospedagem	Docker (Render), com o banco PostgreSQL no Neon

Estrutura

```
src/main/java/br/com/sweetbites
├── model/        entidades JPA (Usuario, Cupcake, Pedido, ItemPedido, Pagamento, Entrega...)
├── repository/   acesso ao banco (Spring Data JPA)
├── service/      regras de negócio (pedido, pagamento, frete, catálogo, usuário)
├── controller/   controllers MVC do cliente e do admin, e API JSON usada pelo JavaScript
├── config/       segurança, dados iniciais, configuração geral
└── job/          tarefa agendada (expiração do PIX)
src/main/resources
├── templates/    telas (Thymeleaf)
├── static/       CSS e JavaScript
└── db/migration/ scripts SQL do banco (Flyway)
src/test/java     testes unitários e de integração
```

Como executar localmente

Pré-requisitos: Java 17 e Docker.

```
docker compose up -d                    # sobe o PostgreSQL na porta 5433
cp .env.example .env                    # ajuste ADMIN_EMAIL e ADMIN_SENHA
set -a; . ./.env; set +a
./mvnw spring-boot:run                  # http://localhost:8080
```

No primeiro boot, o sistema cria as tabelas, o cardápio de exemplo (10 cupcakes em 5 categorias) e o usuário administrador, com o e-mail e a senha definidos em `ADMIN_EMAIL` / `ADMIN_SENHA`.

Para rodar os testes, que usam banco H2 em memória e não precisam de Docker:

```
./mvnw test
```

Manual de uso

Cliente

1. **Criar conta** (US-01): toque em *Cadastre-se* e informe nome, e-mail, telefone e senha (mínimo de 8 caracteres, com letras e números).
2. **Entrar** (US-02): use e-mail e senha. Depois de 5 tentativas erradas a conta é bloqueada; para desbloquear, use *Esqueci minha senha*. Para trocar a senha estando logado, use **Perfil → Alterar senha**.
3. **Recuperar senha** (US-18): informe o e-mail e use o link recebido, válido por 1 hora. Ao redefinir, todas as sessões abertas são encerradas.
4. **Vitrine** (US-04/05): filtre por categoria ou busque por nome ou ingrediente. A busca acontece enquanto você digita. Cupcakes sem estoque aparecem como *Indisponível*.
5. **Detalhes** (US-06): toque no cupcake para ver descrição, ingredientes e preço, escolher a quantidade e adicionar ao carrinho.
6. **Carrinho** (US-07): altere quantidades (máximo de 20 por item) ou remova itens. O carrinho continua salvo mesmo fechando o navegador.
7. **Finalizar pedido** (US-08/09): informe o CEP (o endereço e o frete são calculados automaticamente), o número e a forma de pagamento: cartão (até 3x sem juros), PIX ou dinheiro na entrega. O pedido mínimo é de R$ 15,00.
8. **Acompanhar** (US-10): a tela do pedido mostra o status em tempo real (atualiza sozinha) e cada mudança gera uma notificação no sino do topo.

Administrador

Entre com o e-mail de administrador. Você cai direto no **Painel**.

- **Painel**: pedidos por status e alerta de estoque baixo (menos de 5 unidades).
- **Pedidos** (US-16): filtre por status e período. Abra o pedido para avançar o status (*Em preparação → Saiu para entrega → Entregue*) ou cancelar. O cliente é notificado a cada mudança. Pedidos entregues não podem ser cancelados.
- **Cupcakes** (US-14): cadastre, edite ou remova cupcakes, com foto em JPG/PNG de até 5MB. Cupcakes em pedidos em andamento não podem ser removidos.
- **Estoque** (US-15): ajuste a quantidade disponível. O estoque baixa sozinho a cada pedido e volta quando um pedido é cancelado.
- **Categorias**: crie ou remova categorias (só é possível remover categorias sem cupcakes).

Integrações simuladas (modo demonstração)

Para o projeto acadêmico não ter custo, as integrações pagas são **simuladas**. Com `MODO_DEMONSTRACAO=true`, a interface avisa sempre que algo é simulação.

Integração	Como funciona aqui
Consulta de CEP	**Real**: API pública ViaCEP

Integração	Como funciona aqui
Cartão de crédito	Gateway simulado: o número precisa ser válido (algoritmo de Luhn) e a validade futura. O cartão de teste `4111 1111 1111 1111` é aprovado; cartões terminados em `0002` (ex.: `4000 0000 0000 0002`) são recusados, o que demonstra o fluxo alternativo Beta do UC-08. Só os 4 últimos dígitos são guardados.
PIX	QR Code e código "copia e cola" fictícios, válidos por 30 minutos. O botão *Simular pagamento do PIX* confirma o pagamento. Sem pagamento, o pedido é cancelado automaticamente e o estoque volta.
E-mail (boas-vindas, recuperação de senha, pedido)	Registrado no log do servidor. Na recuperação de senha, o link aparece na tela.
Notificação push	Notificações dentro do app (ícone de sino), com a tela do pedido atualizando o status a cada 10 segundos.

Regras de frete (configuráveis em `application.properties`)

A loja fica em São Paulo/SP. A distância é aproximada por zona:

Destino	Frete	Tempo estimado
Cidade de São Paulo	R$ 7,00	45 min
Outras cidades de SP	R$ 15,00	120 min
Outros estados	Sem entrega	—

Variáveis de ambiente (produção)

Variável	Descrição
`DATABASE_URL`	URL JDBC do PostgreSQL, ex.: `jdbc:postgresql://host/banco?sslmode=require`
`DATABASE_USER` / `DATABASE_PASSWORD`	Credenciais do banco
`ADMIN_EMAIL` / `ADMIN_SENHA`	Administrador criado no primeiro boot
`MODO_DEMONSTRACAO`	`true` exibe os atalhos de simulação (PIX e link de senha)
`PORT`	Porta HTTP (o Render define sozinho)

Segurança

- Senhas guardadas com BCrypt. Bloqueio após 5 tentativas erradas.
- Proteção CSRF em todos os formulários, e Content-Security-Policy sem scripts inline.
- Áreas `/admin/**` liberadas só para o papel ADMIN.
- Um cliente não acessa pedido de outro: a resposta é 404, igual a um pedido inexistente.
- Preços e estoque são sempre recalculados no servidor; o carrinho do navegador não é confiável.
- Upload de imagem validado pelo conteúdo do arquivo, não só pela extensão.
- Token de recuperação de senha guardado como hash SHA-256, de uso único e com validade de 1 hora.

Parte 3: Testes, validação e qualidade (Situação-Problema 3)

Situação-Problema 3: Testes, validação e qualidade

#	Documento	Conteúdo
1	Plano de testes	Estratégia, níveis de teste, ambiente e critérios
2	Testes automatizados	56 testes JUnit, rastreabilidade com os requisitos e testes manuais
3	Roteiro para testadores	Tarefas e dados de teste para os colegas
4	Formulário de feedback	Formulário em três formatos: GitHub Issues, Word e texto
5	Resultados dos testes de aceitação	Roteiro completo em 5 rodadas: 54/54 verificações e 9 problemas encontrados (BUG-06 a 14)
6	Laudo de qualidade	14 defeitos encontrados e corrigidos em 2 rodadas, com evidências de antes e depois

Os prints dos testes ficam em `evidencias/`.

🎬 **Vídeo da solução atualizada (correções BUG-06 a BUG-14, 2min22s):** `video/video-correcoes-bugs.mp4`

1. Plano de testes

Objetivo

Mostrar que o sistema faz o que as histórias de usuário pedem (**verificação**) e que atende quem vai usá-lo (**validação**).

Estratégia

Nível	O que testa	Como	Quem
Unitário	Regras de negócio isoladas (estoque, pagamento, frete, bloqueio de conta, transições de status)	JUnit 5 + Mockito, sem banco	Automatizado
Integração	Aplicação inteira: telas MVC, Spring Security, JPA e Flyway, com banco H2	Spring MockMvc	Automatizado
Sistema (manual)	Fluxos completos no navegador, com layout de celular	Roteiro executado pelo desenvolvedor	Desenvolvedor
Aceitação	Uso real por pessoas que não conhecem o código	Roteiro + formulário	5 colegas

Ambiente

Item	Valor
Testes automatizados	`./mvnw test`, com banco H2 em memória (modo PostgreSQL) e as mesmas migrations de produção
Teste manual	Aplicação local + PostgreSQL 16 (Docker), navegador com tela de 375×812 (celular) e 1280×800 (desktop)
Testes dos colegas	Link público da aplicação (Render), celular ou computador do testador

Critérios de aceite da entrega

1. 100% dos testes automatizados passando.
2. Todos os critérios de aceitação das histórias das Sprints 1 a 3 verificados.
3. Nenhum defeito de severidade **alta** em aberto.
4. Feedback de 5 testadores registrado, com cada item classificado como *corrigido*, *planejado* ou *não aplicável*.

Severidade dos defeitos

Severidade	Definição
Alta	Impede um fluxo principal (comprar, pagar, entrar) ou gera dado errado (valor, estoque)
Média	Funciona, mas com resultado incorreto numa situação específica, ou erro técnico exibido ao usuário
Baixa	Problema visual ou de texto que não impede o uso

2. Testes automatizados (verificação)

Resultado da execução em 10/11/2026: 56 testes, **56 aprovados**, 0 falhas, 0 erros (`./mvnw test`, cerca de 25 s).

```
[INFO] Tests run: 56, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Para rodar:

```
cd situacao-2-codificacao
./mvnw test
```

Rastreabilidade: teste → requisito

Testes unitários (37)

Classe de teste	Teste	Requisito verificado
`UsuarioTest`	bloqueiaContaNaQuintaTentativaErrada	US-02 RN#1
	loginComSucessoZeraTentativas	US-02
	redefinirSenhaDesbloqueiaConta	US-18 / US-02 RN#2
`StatusPedidoTest`	fluxoNormalDePreparacaoEEntrega	US-16 RN#2
	pedidoEntregueOuCanceladoNaoPodeSerCancelado	US-16 RN#1
	ordemDasConstantesEhALinhaDoTempoExibidaAoCliente	US-10 CA#1
`PagamentoServiceTest`	aprovaCartaoValidoEGuardaSoOsUltimos4Digitos	US-09 RN#3
	recusaCartaoTerminadoEm0002	UC-08 curso Beta
	rejeitaNumeroQueNaoPassaNoLuhn	US-09
	rejeitaCartaoVencido / aceitaCartaoQueVenceNoMesCorrente	US-09
	parcelamentoMaximoDe3xSemJuros	US-09 RN#1
	codigoPixIdentificaOPedido	US-09 CA#2
`FreteServiceTest`	mesmaCidadeDaLojaIgnorandoAcentos	US-08 RN#3
	outraCidadeDoMesmoEstado	US-08 RN#3
	outroEstadoNaoTemEntrega	US-08 RN#3
	cepInexistenteOuMalformado	UC-08 curso Alfa
`CarrinhoServiceTest`	interpretaTextoDoNavegadorIgnorandoEntradasInvalidas	US-07 (robustez)
	recalculaPrecosNoServidorELimitaA20PorItem	US-07 RN#1 e RN#3
	sinalizaPedidoAbaixoDoMinimoEItemSemEstoque	US-08 RN#2, US-04 RN#1
	removeItemQueSaiuDoCardapio	US-14 RN#2
`ImagemValidatorTest`	reconhecePngEJpegPelosBytes	US-14 RNF#1

Classe de teste	Teste	Requisito verificado
	rejeitaArquivoDisfarcadoDeImagem	US-14 RNF#1 (segurança)
	rejeitaImagemMaiorQue5MB	US-14 RNF#1
`PedidoServiceTest`	pedidoComCartaoAprovadoVaiParaPreparacaoEBaixaEstoque	US-08, US-15 RN#1
	cartaoRecusadoNaoGravaPedidoNemMexeNoEstoque	UC-08 curso Beta
	pixFicaAguardandoPagamentoComValidadeDe30Minutos	US-09 RN#2
	pedidoMinimoDe15Reais	US-08 RN#2
	naoVendeMaisDoQueOEstoque	UC-08 curso Gama
	maximoDe20UnidadesPorItem	US-07 RN#1
	carrinhoVazio	UC-08 pré-condição
	adminAvancaStatusSomenteNoFluxoENotificaCliente	US-16 CA#2 e RN#2
	entregaEmDinheiroAprovaOPagamento	US-16 RN#3
	pedidoEntregueNaoPodeSerCancelado	US-16 RN#1
	cancelarDevolveEstoqueEEstornaPagamentoAprovado	US-19 RN#1, US-15
	pixVencidoCancelaPedidoEDevolveEstoque	US-09 RN#2
	clienteNaoAcessaPedidoDeOutroCliente	RNF-G3 (segurança)

Testes de integração (10): `FluxosWebTest`

Sobem a aplicação inteira (controllers, segurança, banco com Flyway). Só a API de CEP é substituída por um dublê de teste.

Teste	O que verifica	Requisito
visitanteEhRedirecionadoParaOLogin	Páginas protegidas exigem login	US-04
cadastroValidaSenhaEEmailUnico	Senha fraca recusada; e-mail duplicado (mesmo com maiúsculas) recusado	US-01 CA#1 e CA#2
contaBloqueadaAposCincoTentativasErradas	5ª tentativa errada bloqueia; senha certa não entra depois	US-02 RN#1
recuperarSenhaDesbloqueiaEPermiteNovoLogin	Fluxo completo de redefinição; token não pode ser reutilizado	US-18
emailInexistenteRecebeAMesmaRespostaNaRecuperacao	Não revela quem tem conta	US-18 RN#2
clienteNaoAcessaPainelAdmin	Cliente recebe 403 em `/admin`; admin acessa e filtra pedidos	US-14 a US-16
buscaNaVitrineIgnoraMaiusculasEProcuraIngredientes	"MORANGO" encontra por nome e ingrediente; mostra "Produto não encontrado"; mostra "Indisponível"	US-05, US-04 RN#1
pedidoCompletoComPixEOwnership	Compra via PIX, baixa de estoque, QR Code, outro cliente recebe 404, pagamento simulado	US-08, US-09, US-10, RNF-G3
checkoutRejeitaPedidoAbaixoDoMinimoMostrandoAMensagem	Mensagem do pedido mínimo na tela	US-08 RN#2

Teste	O que verifica	Requisito
apiDeFreteDevolveErroAmigavelParaCepInvalido	CEP inválido → erro amigável; CEP de SP → R$ 7,00 / 45 min	UC-08 Alfa, US-08 RN#3

Testes de regressão da 2ª rodada (9): `CorrecoesRodada2Test`

Teste	Defeito coberto
bug06SessaoExpiradaAoEnviarFormularioLevaAoLoginComAviso	BUG-06
bug07ApiSemSessaoResponde401ParaOJavaScript	BUG-07
bug08ClienteAlteraSenhaPeloPerfil	BUG-08 (senha atual errada, nova igual à atual, sucesso e login com a nova)
bug09LayoutInformaOUsuarioParaSepararOCarrinho	BUG-09
bug10StatusDoPagamentoNoAdminEmPortugues	BUG-10
bug11FiltroComPeriodoInvertidoAvisaOAdmin	BUG-11
bug12IdInvalidoNaUrlMostraPaginaNaoEncontrada	BUG-12
bug13UsuarioLogadoNaoVeTelaDeLoginDeNovo	BUG-13
bug14ImagemEntre5e10MBRecebeMensagemAmigavel	BUG-14

Testes manuais de sistema (desenvolvedor)

Executados no navegador com tela de celular (375×812). As evidências estão em `evidencias/`.

#	Cenário	Resultado	Evidência
M01	Cadastro com campos vazios ou senha fraca	✅ Mensagens por campo	bug05-corrigido-uma-mensagem.jpg
M02	Login e vitrine com saudação	✅	—
M03	Busca "MORANGO" em tempo real	✅ 2 resultados (nome e ingrediente)	—
M04	Carrinho com 2 itens e subtotal	✅	—
M05	Checkout com CEP 01310-100 (Av. Paulista)	✅ após a correção do BUG-01	bug01-corrigido-frete-45min.jpg
M06	Cartão 4000 0000 0000 0002	✅ Recusado, nada gravado, endereço mantido	teste-cartao-recusado.jpg
M07	Pedido via PIX + simular pagamento	✅ "Em preparação"	teste-pix-pago-em-preparacao.jpg
M08	Admin muda o status; a tela do cliente atualiza sozinha	✅ em menos de 10 s, com notificação	teste-status-tempo-real.jpg
M09	Pedido com cartão 4111…	✅ Confirmado, carrinho esvaziado	bug02-03-corrigido-confirmacao.jpg
M10	Upload de arquivo falso como imagem	✅ Recusado: "Formato de imagem não suportado"	—
M11	Admin tenta voltar o status de "Saiu para entrega" para "Em preparação"	✅ Recusado com mensagem	—

#	Cenário	Resultado	Evidência
M12	Acesso a `/admin` sem login	✅ Redireciona para o login	—
M13	CEP do Rio de Janeiro	✅ "Ainda não entregamos em Rio de Janeiro/RJ"	—

3. Roteiro para os testadores (validação com usuários)

Olá! Obrigado por testar a **Sweet Bites**, uma loja virtual de cupcakes. O teste leva uns 15 minutos. Use, de preferência, o **celular**.

Link da aplicação: *(será preenchido quando o sistema estiver no ar)* · O servidor gratuito "dorme" quando ninguém usa: **o primeiro acesso pode levar até 1 minuto**.

> Pagamentos, PIX e e-mails são **simulados**: nada é cobrado de verdade. Não use dados reais de cartão.

Dados de teste

Para	Use
Cartão aprovado	`4111 1111 1111 1111`, validade `12/30`, CVV `123`
Cartão recusado	`4000 0000 0000 0002`, validade `12/30`, CVV `123`
CEP em São Paulo (frete R$ 7,00)	`01310-100` (Av. Paulista)
CEP em outra cidade de SP (frete R$ 15,00)	`13010-000` (Campinas)
CEP fora da área de entrega	`20040-020` (Rio de Janeiro)

Tarefas

Faça as tarefas na ordem e anote no formulário o que funcionou, o que não funcionou e o que você não conseguiu testar. **Tire print** de qualquer erro.

Parte A: Cliente

1. **Crie uma conta.** Primeiro tente uma senha fraca (ex.: `abc`) e veja a mensagem. Depois use uma senha válida.
2. **Entre** com a conta criada.
3. Na vitrine, **filtre** pela categoria "Chocolate" e depois volte para "Todos".
4. **Busque** "morango" e confira se aparecem cupcakes com morango no nome ou nos ingredientes.
5. Abra um cupcake **indisponível** (ex.: "Nutella com Avelã") e confira se não dá para comprá-lo.
6. Abra um cupcake, escolha **3 unidades** e adicione ao carrinho.
7. No **carrinho**, aumente, diminua e remova itens. Feche o navegador, abra de novo e confira se o carrinho continua lá.
8. Deixe no carrinho **menos de R$ 15,00** e tente finalizar. Deve aparecer o aviso de pedido mínimo.
9. Complete o carrinho e vá em **Finalizar pedido**. Teste o CEP fora da área e depois um CEP válido.
10. Pague com o **cartão recusado**: deve aparecer "Pagamento não aprovado".
11. Pague com o **cartão aprovado**: deve aparecer "Pedido confirmado!".
12. Faça outro pedido com **PIX**, copie o código e use **Simular pagamento do PIX**.
13. Abra **Meus pedidos** e o **sino de notificações**.
14. Cancele um pedido que esteja "Em preparação".
15. Use **Esqueci minha senha**, redefina a senha pelo link exibido e entre com a nova senha.
16. Saia da conta (**Perfil → Sair**).

Parte B: Administrador *(só se você recebeu o login de admin)*

17. Veja o **painel** e o alerta de estoque baixo.
18. Cadastre um **novo cupcake** com foto (JPG ou PNG). Tente também enviar um arquivo que não seja imagem.
19. Altere o **estoque** de um cupcake para 0 e confira na vitrine se ele aparece como "Indisponível".
20. Em **Pedidos**, abra um pedido e avance o status até "Entregue". Na conta de cliente, veja a notificação. Tente cancelar o pedido entregue.

Parte C: Opinião

- O que achou do visual e da facilidade de uso?
- O que você mudaria ou acrescentaria?

4. Formulário de feedback dos testadores

O material pede um formulário para registrar **bugs, sugestões de melhoria e prints das telas com erro**. Há três formas de enviar, escolha a mais fácil para cada colega:

1. **GitHub Issues (preferencial):** em Issues → New issue, escolha 🐞 **Reportar um bug** ou 💡 **Sugerir melhoria**. É possível arrastar prints direto no formulário.
2. **Arquivo Word:** preencha `formulario-de-feedback.docx` e devolva por e-mail ou WhatsApp, com os prints.
3. **Texto:** copie o modelo abaixo numa mensagem.

Modelo

Nome: Data do teste: Aparelho e navegador: (ex.: Android + Chrome, iPhone + Safari, notebook + Edge)

1. O que você testou e funcionou

(Use os números das tarefas do roteiro, ex.: "1 a 9 funcionaram".)

2. O que você testou e NÃO funcionou (bugs)

Para cada problema:

#	Tarefa do roteiro	O que você fez	O que aconteceu	O que esperava	Print anexado?
1					
2					

3. Funcionalidades que você não testou

(Porque faltou tempo, não encontrou ou não estava implementada.)

4. Sugestões de melhoria

5. Avaliação geral (de 1 a 5)

- Facilidade de uso: ☐1 ☐2 ☐3 ☐4 ☐5
- Visual: ☐1 ☐2 ☐3 ☐4 ☐5
- Você compraria por esse app? ☐Sim ☐Talvez ☐Não

5. Resultados dos testes de aceitação

> **Observação:** os colegas convidados não puderam testar a aplicação dentro do prazo. Por isso, o desenvolvedor executou o roteiro de 20 tarefas inteiro em **10/11/2026**, na aplicação local com PostgreSQL, dividido em 5 rodadas por área do sistema.
>
> **Como foi executado:** - as verificações de servidor foram automatizadas com o script `scripts/roteiro_aceitacao.py`, que simula o navegador com sessão, CSRF e envio de formulários; - as partes que dependem de JavaScript foram conferidas manualmente no navegador, com tela de celular: busca enquanto digita, carrinho salvo, aviso de pedido mínimo e atualização do status em tempo real.
>
> **Resultado:** o roteiro automatizado teve **54 de 54 verificações aprovadas** (saída completa em `evidencias/resultados-aceitacao-2026-11-10.json`). A exploração manual das mesmas áreas encontrou **9 problemas (BUG-06 a BUG-14)**, listados em cada rodada abaixo e corrigidos depois (ver a seção 2.1 do laudo). Com as correções, o roteiro foi repetido: 54 de 54.
>
> Se algum colega testar depois, o formulário continua disponível e o retorno dele entra na seção "Testes com colegas", no fim deste documento.

Teste 1: Conta e acesso (tarefas 1, 2, 15 e 16)

Campo	Resultado
Executado por	Bruno Villanova da Silva (desenvolvedor)
Data	10/11/2026
Testou e funcionou	**Cadastro:** senha fraca recusada com mensagem; e-mail repetido (em maiúsculas) recusado; cadastro válido leva ao login com "Conta criada com sucesso". **Login:** saudação pelo nome. **Bloqueio:** após 5 senhas erradas, nem a senha certa entra. **Esqueci minha senha:** link exibido (modo demonstração), conta desbloqueada e link de uso único. **Logout:** volta ao login e a vitrine passa a exigir login.
Não funcionou / corrigir	**BUG-08:** "Alterar senha" no perfil abre o fluxo de "Esqueci minha senha" mesmo com o cliente logado. Corrigir: tela própria com senha atual e nova. **BUG-13:** quem já está logado e abre `/login` ou `/cadastro` vê o formulário de novo. Corrigir: redirecionar para a vitrine.
Não testado	Recebimento real de e-mail (simulado); login social (fora do escopo)

Teste 2: Vitrine e carrinho (tarefas 3 a 8)

Campo	Resultado
Executado por	Bruno Villanova da Silva (desenvolvedor)
Data	10/11/2026

Campo	Resultado
Testou e funcionou	**Vitrine:** 10 cupcakes e 5 categorias; filtro "Chocolate" mostra só os 2 de chocolate. **Busca:** acontece enquanto o cliente digita ("lim" → Limão Siciliano); "MORANGO" encontra 2 cupcakes por nome e ingrediente; busca sem resultado mostra "Produto não encontrado". **Indisponível:** Nutella com Avelã aparece sem botão de compra. **Detalhes:** com ingredientes e "Impostos inclusos". **Carrinho:** limite de 20 por item; continua salvo após recarregar; abaixo de R$ 15,00 mostra "Faltam R$ 0,50" e bloqueia a finalização; com R$ 29,00 libera.
Não funcionou / corrigir	**BUG-09:** o carrinho fica no navegador sem dono; a cliente que entrou depois do admin no mesmo navegador encontrou o item dele no carrinho. Corrigir: carrinho separado por usuário. **BUG-12:** `/cupcakes/abc` mostra "Algo deu errado". Corrigir: página "Não encontramos" (404).
Não testado	Favoritos e avaliações (Sprint 4)

Teste 3: Checkout e pagamento (tarefas 8 a 12)

Campo	Resultado
Executado por	Bruno Villanova da Silva (desenvolvedor)
Data	10/11/2026
Testou e funcionou	**CEP:** do Rio de Janeiro e inexistente recusados. **Frete:** São Paulo R$ 7,00 / 45 min; Campinas R$ 15,00 / 120 min. **Validações:** pedido abaixo de R$ 15,00 e quantidade acima do estoque recusados. **Cartão recusado** (0002): estoque não muda (30 → 30). **Cartão aprovado** (4111, em 2x): "Pedido confirmado!" e estoque baixou de 30 para 28. **PIX:** QR Code e código gerados; "Simular pagamento" leva o pedido para Em preparação.
Não funcionou / corrigir	**BUG-06:** com a sessão encerrada, "Confirmar pedido" mostra "Acesso negado – você não tem permissão". Corrigir: levar ao login com o aviso de sessão encerrada.
Não testado	Gateway real (simulado); expiração do PIX após 30 min (coberta pelo teste automatizado `pixVencidoCancelaPedidoEDevolveEstoque`); endereços salvos (Sprint 4)

Teste 4: Acompanhamento e cancelamento (tarefas 13 e 14)

Campo	Resultado
Executado por	Bruno Villanova da Silva (desenvolvedor)
Data	10/11/2026
Testou e funcionou	**Meus pedidos:** lista com número, data, pagamento, valor e status. **Notificações:** sino com 3 não lidas e uma notificação por etapa. **Tempo real:** a tela do pedido recarregou sozinha em menos de 10 s após a mudança feita pelo admin. **Cancelamento:** devolve o estoque (Pistache 11 → 12), e um pedido já cancelado não pode ser cancelado de novo.
Não funcionou / corrigir	**BUG-07:** se a sessão expira com a tela do pedido aberta, a atualização do status para sem aviso. Corrigir: avisar e levar ao login.
Não testado	Push no celular (substituído por notificação no app); "Repetir pedido" (Sprint 4)

Teste 5: Administrador e segurança (tarefas 17 a 20)

Campo	Resultado
Executado por	Bruno Villanova da Silva (desenvolvedor)
Data	10/11/2026
Testou e funcionou	**Painel:** contadores e alerta de estoque baixo. **Cupcakes:** arquivo falso recusado; novo cupcake com PNG aparece na vitrine; estoque 0 deixa o cupcake Indisponível; cupcake em pedido ativo não pode ser removido. **Pedidos:** não é possível pular etapa; o cliente vê "Saiu para entrega" e "Entregue", com notificação; pedido entregue não pode ser cancelado. **Segurança:** cliente recebe 403 no painel admin e 404 no pedido de outro cliente.
Não funcionou / corrigir	**BUG-10:** status do pagamento aparece como "APROVADO". Corrigir: descrição em português. **BUG-11:** filtro com período invertido mostra lista vazia sem explicação. Corrigir: avisar. **BUG-14:** imagem de 5 a 10MB enviada sem o JavaScript dá erro genérico 413. Corrigir: mensagem do limite de 5MB.
Não testado	Relatórios em PDF (Sprint 4); upload acima de 5MB pela tela (coberto pelo teste automatizado `rejeitaImagemMaiorQue5MB`)

Como repetir o teste

Com a aplicação rodando (ver o README do código):

```
python3 situacao-3-testes-validacao/scripts/roteiro_aceitacao.py http://localhost:8080
<email-admin> <senha-admin> resultado.json
```

Testes com colegas

Nome	Data	Funcionou	Não funcionou	Não testado	Decisão

6. Laudo de qualidade

Item	Informação
Sistema	Sweet Bites: loja virtual de cupcakes gourmet
Versão avaliada	Branch `main` do repositório Brunoredbov/sweet-bites
Data	10/11/2026
Responsável	Bruno Villanova da Silva
Ambiente	Java 17 + Spring Boot 4, PostgreSQL 16 (Docker); navegador com tela de celular 375×812 e de desktop 1280×800

1. Resumo

Indicador	Resultado
Testes automatizados	**56 de 56 aprovados** (37 unitários + 19 de integração)
Histórias das Sprints 1 a 3	17 de 17 implementadas (4 com ajustes documentados)
Cenários manuais executados	13 de 13 aprovados, após as correções
Roteiro de aceitação (20 tarefas)	54 de 54 verificações aprovadas
Defeitos encontrados	**14**: 5 na 1ª rodada (3 altos, 1 médio, 1 baixo) e 9 na 2ª rodada (3 médios, 6 baixos)
Defeitos corrigidos e retestados	**14 de 14**
Defeitos em aberto	0

Parecer: o sistema está **apto para os testes com usuários (validação)**. Os pontos da seção 4 são limitações conhecidas e documentadas, não defeitos.

2. Defeitos encontrados e correções

BUG-01: Frete da cidade de São Paulo calculado como "outra cidade" · Severidade Alta

- **Como foi encontrado:** teste manual M05, checkout com o CEP 01310-100 (Av. Paulista).
- **Comportamento:** o sistema mostrava "entrega estimada em 120 min" e cobrava **R$ 15,00**, quando o certo seria 45 min e R$ 7,00. Ou seja, cobrava a mais de todo cliente da capital.
- **Causa:** o arquivo `application.properties` é lido pelo Java na codificação ISO-8859-1. O nome `São Paulo`, salvo em UTF-8, virou `SÃ£o Paulo` e nunca era igual à cidade devolvida pelo ViaCEP.
- **Correção:** o valor foi escrito com escape Unicode (`São Paulo`), e o teste unitário passou a cobrir a comparação com e sem acento.
- **Reteste:** ✅ R$ 7,00 / 45 min. Também coberto pelos testes `FreteServiceTest.mesmaCidadeDaLojaIgnorandoAcentos` e `FluxosWebTest.apiDeFreteDevolveErroAmigavelParaCepInvalido`.
- **Evidências:** antes, bug01-frete-sao-paulo-120min.jpg; depois, bug01-corrigido-frete-45min.jpg.

BUG-02: Carrinho não era esvaziado depois do pedido confirmado · Severidade Média

- **Como foi encontrado:** teste manual M07. Depois de confirmar o pedido, o ícone do carrinho continuava mostrando "2".
- **Risco:** o cliente poderia comprar os mesmos itens de novo sem perceber.
- **Causa:** a tela marcava o pedido recém-confirmado com o atributo `data-limpar-carrinho=""`, mas o Thymeleaf **remove atributos com valor vazio**, então o JavaScript nunca recebia o aviso.
- **Correção:** o atributo passou a ter o valor `"true"`.
- **Reteste:** ✅ o carrinho zera na tela de confirmação.
- **Evidências:** antes, bug02-carrinho-nao-esvaziou.jpg; depois, bug02-03-corrigido-confirmacao.jpg.

BUG-03: Erro 500 na tela de acompanhamento do pedido · Severidade Alta

- **Como foi encontrado:** teste manual M07. Logo depois de concluir um pedido via PIX, aparecia a página "Algo deu errado".
- **Causa:** a linha do tempo usava `#lists.indexOf(...)`, uma função que **não existe** no Thymeleaf (*SpelEvaluationException: Method indexOf cannot be found*).
- **Correção:** a etapa atual passou a ser obtida pela ordem do próprio enum (`pedido.status.ordinal()`). Um teste (`StatusPedidoTest.ordemDasConstantesEhALinhaDoTempoExibidaAoCliente`) garante que a ordem não seja alterada por engano.
- **Reteste:** ✅ a tela abre com a linha do tempo correta.
- **Evidências:** antes, bug03-erro-500-linha-do-tempo.jpg; depois, bug02-03-corrigido-confirmacao.jpg.

BUG-04: Erro 500 na lista de pedidos do administrador · Severidade Alta

- **Como foi encontrado:** varredura das telas do admin (`/admin/pedidos`), com e sem filtros.
- **Causa:** a consulta usava parâmetros opcionais no formato `(:inicio IS NULL OR ...)`. O PostgreSQL não consegue deduzir o tipo de um parâmetro nulo (*ERROR: could not determine data type of parameter $3*). Os testes automatizados não pegaram o erro porque rodam no H2, que aceita essa construção.
- **Correção:** os parâmetros passaram a ser sempre preenchidos (lista de status e período padrão de 2000 a 2999). A mesma correção foi aplicada **preventivamente** à consulta da vitrine, que tinha a mesma construção.
- **Reteste:** ✅ `/admin/pedidos` sem filtro, com status e com período: todos com HTTP 200 no PostgreSQL real.
- **Lição aprendida:** testar também contra o banco de produção (PostgreSQL), não só contra o H2.

BUG-05: Duas mensagens de erro no mesmo campo do cadastro · Severidade Baixa

- **Como foi encontrado:** durante a captura das telas para a documentação. Com o campo telefone vazio, apareciam "Informe um telefone válido…" **e** "Informe seu telefone."; com a senha vazia, o mesmo acontecia.
- **Causa:** duas validações (`@NotBlank` e `@Pattern`) falhavam juntas e a tela mostrava todas.
- **Correção:** todas as telas de formulário passaram a mostrar **apenas a primeira mensagem** de cada campo.
- **Reteste:** ✅ uma mensagem por campo.
- **Evidências:** antes, bug05-mensagens-duplicadas.jpg; depois, bug05-corrigido-uma-mensagem.jpg.

2.1 Segunda rodada: revisão de qualidade pós-aceitação (10/11/2026)

Depois do roteiro de aceitação, o sistema foi revisado procurando **casos extremos e problemas de usabilidade**: sessão expirada, troca de usuário no mesmo aparelho, URLs digitadas à mão, filtros incoerentes e arquivos grandes. Os 9 problemas encontrados foram corrigidos, ganharam testes automatizados (`CorrecoesRodada2Test` , 9 testes) e foram retestados no navegador. A versão antiga foi fotografada antes da correção, e a nova, depois.

ID	Problema encontrado	Sev.	Correção	Antes → depois
BUG-06	Com a sessão expirada, enviar um formulário (ex.: confirmar o pedido) mostrava **"Acesso negado – você não tem permissão"**, mensagem errada e assustadora	Média	Token de segurança (CSRF) vencido agora leva ao login com "Sua sessão foi encerrada. Entre novamente."	antes → depois
BUG-07	Sem sessão, a API do acompanhamento respondia com a página de login (HTML), e a tela do pedido **parava de atualizar sem avisar**	Baixa	A API responde 401 e o JavaScript leva o cliente ao login	teste `bug07ApiSemSessaoResponde401ParaOJavaScript`
BUG-08	"Alterar senha" no perfil abria o fluxo de **"Esqueci minha senha"** (pedia e-mail e link) mesmo com o cliente logado	Média	Nova tela **Perfil → Alterar senha** (senha atual + nova senha), avisando quando a senha atual está errada ou a nova é igual à atual	antes → depois
BUG-09	O carrinho ficava salvo no navegador sem dono: quem saía pelo "Sair" do painel (ou tinha a sessão expirada) **deixava o carrinho para o próximo usuário** (a cliente viu o Pistache colocado pelo admin)	Média	Carrinho separado por usuário (chave com o id da conta); a chave antiga, compartilhada, é apagada	antes → depois
BUG-10	No admin, o status do pagamento aparecia como código interno (**"APROVADO"**, "PENDENTE")	Baixa	Status com descrição em português ("Aprovado", "Pendente")	antes → depois

ID	Problema encontrado	Sev.	Correção	Antes → depois
BUG-11	Filtro de pedidos com a data inicial **depois** da final mostrava só "Nenhum pedido encontrado", sem dizer o motivo	Baixa	Aviso: "A data inicial é posterior à data final. Ajuste o período."	antes → depois
BUG-12	URL com código inválido (ex.: `/cupcakes/abc`) mostrava **"Algo deu errado"** (erro 400)	Baixa	Mostra a página "Ops! Não encontramos essa página" (404)	antes → depois
BUG-13	Cliente já logado que abria `/login` ou `/cadastro` via de novo o formulário de entrada	Baixa	Redireciona para a vitrine (ou para o painel, se for admin)	antes → teste `bug13UsuarioLogadoNaoVeTelaDeLoginDeNovo`
BUG-14	Imagem entre 5 e 10MB enviada sem passar pelo JavaScript gerava erro genérico (HTTP 413), porque o limite do servidor estourava antes da validação	Baixa	O limite técnico do servidor subiu para 10MB e a regra de 5MB ficou no validador, com a mensagem "A imagem deve ter no máximo 5MB."; a página de erro também explica o caso 413	teste `bug14ImagemEntre5e10MBRecebeMensagemAmigavel` + reteste com 6MB no servidor real

Lição aprendida: a primeira versão da correção BUG-07 fez **todas** as páginas responderem 401 para requisições sem cabeçalho de navegador. O teste antigo `visitanteEhRedirecionadoParaOLogin` pegou a regressão na hora, e a regra foi restrita a `/api/**`. Depois das correções, o roteiro de aceitação foi executado de novo: **54 de 54**.

3. Regras de negócio verificadas com sucesso

Regra	Evidência
Cartão recusado não grava pedido nem altera estoque (UC-08 curso Beta)	teste-cartao-recusado.jpg + teste `cartaoRecusadoNaoGravaPedidoNemMexeNoEstoque`
PIX pago leva o pedido para "Em preparação"	teste-pix-pago-em-preparacao.jpg
Mudança de status pelo admin aparece para o cliente em menos de 10 s, com notificação	teste-status-tempo-real.jpg
Bloqueio de conta após 5 tentativas	teste `contaBloqueadaAposCincoTentativasErradas`
Cliente não acessa pedido de outro cliente (404)	teste `pedidoCompletoComPixEOwnership`
Upload de arquivo que não é imagem é recusado	teste `rejeitaArquivoDisfarcadoDeImagem` + teste manual M10

4. Limitações conhecidas (não são defeitos)

#	Limitação	Impacto	Plano
L1	Pagamento, PIX, e-mail e push são simulados	Nenhuma cobrança ou e-mail real	Trocar as implementações dos services (ver Arquitetura)
L2	Endereços não ficam salvos na conta	O cliente digita o CEP a cada pedido	Sprint 4
L3	Sessões ficam na memória do servidor	Reiniciar o servidor desconecta os usuários	Aceitável no plano gratuito; sessão no banco se necessário
L4	Hospedagem gratuita "dorme" depois de 15 min sem uso	O primeiro acesso pode levar cerca de 1 minuto	Avisar os testadores no roteiro
L5	US-11, US-12 (repetir pedido), US-13 e US-17 não implementadas	Fora do escopo das Sprints 1 a 3	Sprint 4

5. Testes de aceitação (roteiro completo)

O roteiro de 20 tarefas foi executado pelo desenvolvedor em 10/11/2026 (os colegas convidados não puderam testar no prazo): **54 de 54 verificações automatizadas aprovadas**. A exploração manual das mesmas áreas encontrou os 9 problemas BUG-06 a BUG-14, corrigidos na 2ª rodada (seção 2.1). Depois das correções, o roteiro foi repetido com 54 de 54. Os detalhes de cada rodada estão em Resultados dos testes de aceitação.